

PSG COLLEGE OF TECHNOLOGY

(Autonomous institution)

COIMBATORE-641 004

23LF13 EMBEDDED PROCESSING USING FPGA

Dissertation submitted in partial fulfilment of the requirements for the
degree of

BACHELOR OF ENGINEERING

**BRANCH: ELECTRONICS AND COMMUNICATION
ENGINEERING**



April-2026

Hardware-Accelerated Closed-Loop PID Control System

DEPARMENT OF ELECTRONICS AND COMMUNICATION ENGINEERING

SUBMITTED BY:

ARVIND PRAKASH T (23L210)

MOHIT S (23L243)

SATHISH M (23L261)

Hardware-Accelerated Closed-Loop PID Control System

Hardware/Software Codesign Project Report

Project Title	Hardware-Accelerated Closed-Loop PID Control System
Target Platform	ZedBoard Zynq Evaluation and Development Kit (XC7Z020)
Processor	MicroBlaze-V Soft Processor (RISC-V ISA based, implemented in PL)
Tools Used	Vivado 2025.1 (Hardware Design) + Vitis 2025.1 (Software Development)
Custom IP	PID_Accelerator_IP (AXI4-Lite Slave with dedicated 32-bit DSP arithmetic)
Communication	JTAG terminal
Local Memory	64 KB BRAM (Dual-port Block RAM) for high-speed instruction/data storage

Co-design is the process of designing **hardware and software together at the same time** using a common system description.

Objectives

- **Coordination** → Hardware and software work properly together
- **Concurrency** → Designed in parallel (saves time)
- **Correctness** → Early testing reduces errors
- **Complexity** → Handles complex systems easily

Motivation

- Reduces **time to market**
- Improves **performance and reliability**
- Fix hardware issues **before fabrication**
- Useful for **SoC and embedded systems**
- Faster and better **integration**

Need

- Systems use both **hardware + software**
- Use of **processors is increasing**
- Helps in **hardware vs software decisions (trade-off)**

Features

- Hardware & software designed **together**
- **Fast verification**
- **Easy design exploration**

Project Description:

Introduction:

This project details the development of a Hardware-Accelerated Closed-Loop PID Control System deployed on a Xilinx Zynq-7000 System-on-Chip (SoC). By leveraging the unique capabilities of the Zynq platform, the project creates a robust, real-time cruise control simulation that bridges the gap between high-level software logic and low-level deterministic hardware acceleration.

Project Overview:

At its core, this system addresses a classic engineering challenge: achieving precise control in a dynamic, time-sensitive environment. While typical cruise control systems rely on complex software stacks that can be susceptible to latency and jitter, this implementation offloads the time-critical PID (Proportional-Integral-Derivative) control calculations directly into the FPGA fabric. This "Hardware/Software Co-Design" allows the system to maintain a deterministic execution cycle, ensuring that the control effort remains stable regardless of the workload handled by the ARM processor.

Key Objectives:

- **Deterministic Control:** Migrating the PID algorithm from the ARM Cortex-A9 processor to an AXI-mapped hardware IP core to eliminate OS-level timing inconsistencies.
- **Integrated Simulation:** Developing a physics-based model that mimics real-world automotive dynamics, including momentum and drag, to provide a realistic testing environment.
- **System Reliability:** Ensuring a stable performance profile through managed power distribution and robust peripheral drivers, moving beyond simple prototyping to a production-grade reliable system.

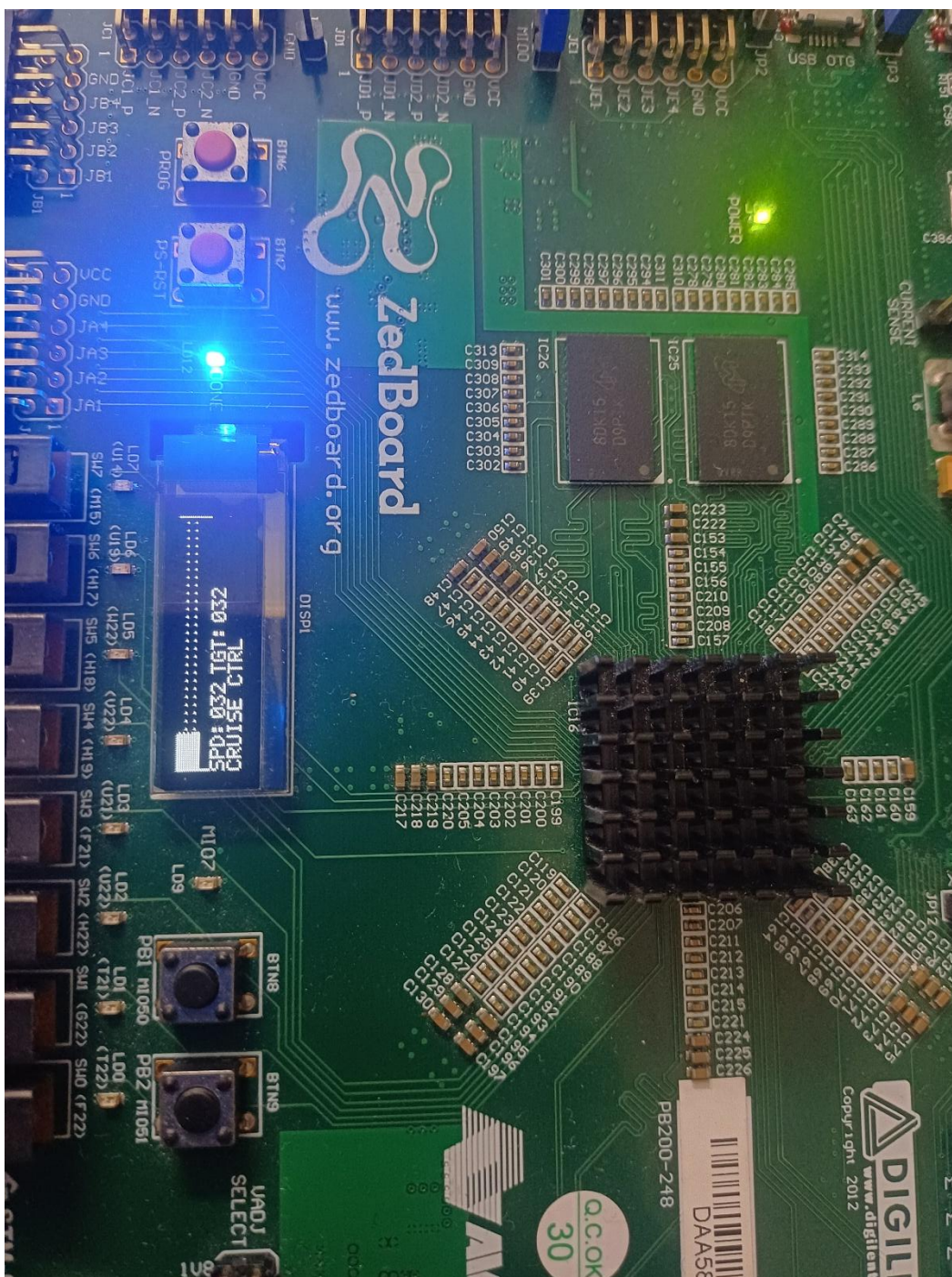
The role of Hardware Software Codesign:

The project utilizes the Zynq SoC's heterogeneous architecture to optimize task allocation:

- **FPGA Partition:** Functions as the "deterministic engine," executing the high-speed PID mathematical loops to guarantee instant responsiveness to sensor inputs.
- **ARM Partition:** Functions as the "system supervisor," managing complex but less time-critical tasks such as human-machine interface (HMI) logic, data visualization on an OLED display, and telemetry transmission to a host computer.

By decoupling the high-speed control loop from the secondary diagnostic and monitoring tasks, the system achieves a sophisticated balance of performance and accessibility, demonstrating a mastery of embedded systems design, digital logic implementation, and real-time Linux-based peripheral management.

Results:



As shown in the image, the display provides the user with an immediate visual confirmation of the system's state:

- **SPD (Current Speed):** Displays the current velocity calculated by the PID algorithm in the FPGA fabric.
- **TGT (Target Speed):** Reflects the value set by your slide switches.
- **Bar Gauge:** Offers a dynamic, intuitive representation of how the PID controller's effort is currently working to close the gap between current and target speeds.

This display is crucial because it proves that even with high-speed, 50ms-interval control calculations happening in the background, the system successfully manages to synchronize and visualize data for the user without performance degradation.

Working:

To understand how your cruise control system reaches the target speed, it helps to visualize the PID (Proportional-Integral-Derivative) controller as a mechanism that continuously minimizes the **Error**, defined as the difference between your **Target Speed** and your **Current Speed** ($e = T - S$).

When you adjust the slide switches on your ZedBoard, you are setting a new Target Speed. The system then enters a closed-loop cycle where the FPGA calculates the necessary control effort to bridge that gap.

How the PID Components Drive the System to Target:

To understand how your cruise control system reaches the target speed, it helps to visualize the PID (Proportional-Integral-Derivative) controller as a mechanism that continuously minimizes the **Error ()**, defined as the difference between your **Target Speed ()** and your **Current Speed ()**.

When you adjust the slide switches on your ZedBoard, you are setting a new Target Speed. The system then enters a closed-loop cycle where the FPGA calculates the necessary control effort to bridge that gap.

How the PID Components Drive the System to Target

Each component of the PID algorithm contributes differently to how the system responds:

- **Proportional (): The "Immediate Correction"**
 - **Action:** It produces an output proportional to the current error. If your target is 032 and you are currently at 005, the error is large (27), resulting in a high initial control effort to accelerate rapidly.
 - **Behavior:** As the Current Speed () approaches the Target Speed (), the error decreases, and the Proportional action naturally tapers off. This is why you saw rapid attainment after increasing to 15—it provides the "oomph" needed to start moving.

- **Integral (): The "Steady-State Eliminator"**

- **Action:** It accumulates the error over time. If your system is slightly underperforming due to simulated drag (physics simulation), the Proportional term might not be enough to reach the exact target, leaving a small, persistent error. The Integral term "remembers" this and grows larger the longer the error exists, gradually forcing the system to reach the exact target speed.
- **Behavior:** It ensures that "SPD: 032" on your OLED display perfectly matches "TGT: 032" without settling at a slightly lower value due to friction.

- **Derivative (): The "Damping Factor"**

- **Action:** It monitors the rate of change of the error. It acts as a "brake" or a dampener.
- **Behavior:** If the system is approaching the target speed too quickly (which might cause the speed to "overshoot" 032 and go to 035), the Derivative term senses this rapid change and reduces the control effort, allowing the system to settle smoothly into the target speed.

Visualizing the Dynamic Response:

When you change the switches, your system is performing a Step Response. Here is how the values reach the target:

1. **Rise Phase (Dominantly Proportional):** The moment you change the switch, the system sees a large error. The FPGA logic applies maximum control effort to accelerate, causing the bar gauge on your OLED to fill up rapidly.
2. **Settling Phase (Integral & Derivative Interaction):** As the speed approaches 032, the Proportional effort decreases. The Derivative term prevents the speed from jumping past 032, while the Integral term works to ensure that the final reading is rock-solid at the target without "drooping" due to the simulated drag you implemented.

2. Project Design

In this project, a custom AXI IP block is created in Vivado and its functionality is defined by integrating custom Verilog code. The hardware platform used here is the **ZedBoard Zynq-7000 FPGA board** and a **MicroBlaze-V soft processor** running in the Programmable Logic (PL).

In this project, a custom LED controller IP core is created where reads and writes are performed over an AXI (AMBA Extensible Interface) bus by the MicroBlaze-V processor core. The processor sends mode and data values to the hardware IP through AXI registers, and the hardware drives 8 onboard LEDs accordingly.

Hardware/Software Codesign Requirements:

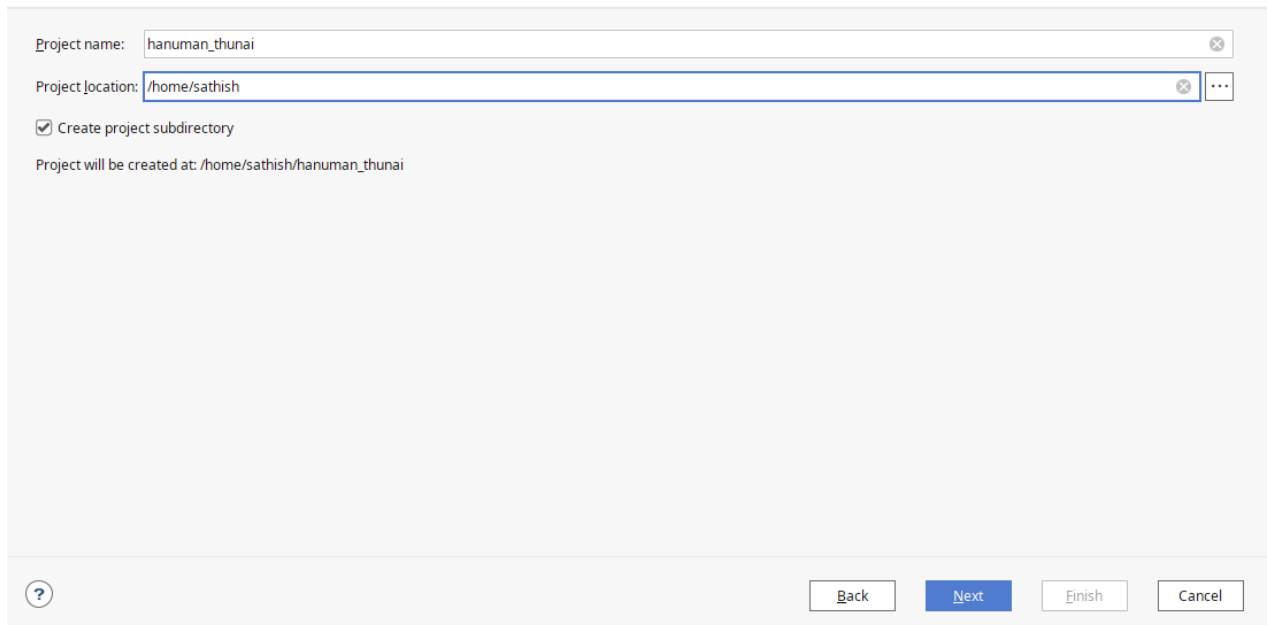
- Boards: ZedBoard Zynq-7000
- Vivado 2025.1 and Vitis 2025.1

Creating the Project in Vivado

1. Start the **Vivado 2025.1** application. In the startup screen, select **New Project** and click "Next". Enter the project details as follows: **Project name:** MicroBlaze-V_SoC **Project location:** User defined directory or default location. Tick the "Create project subdirectory" checkbox.

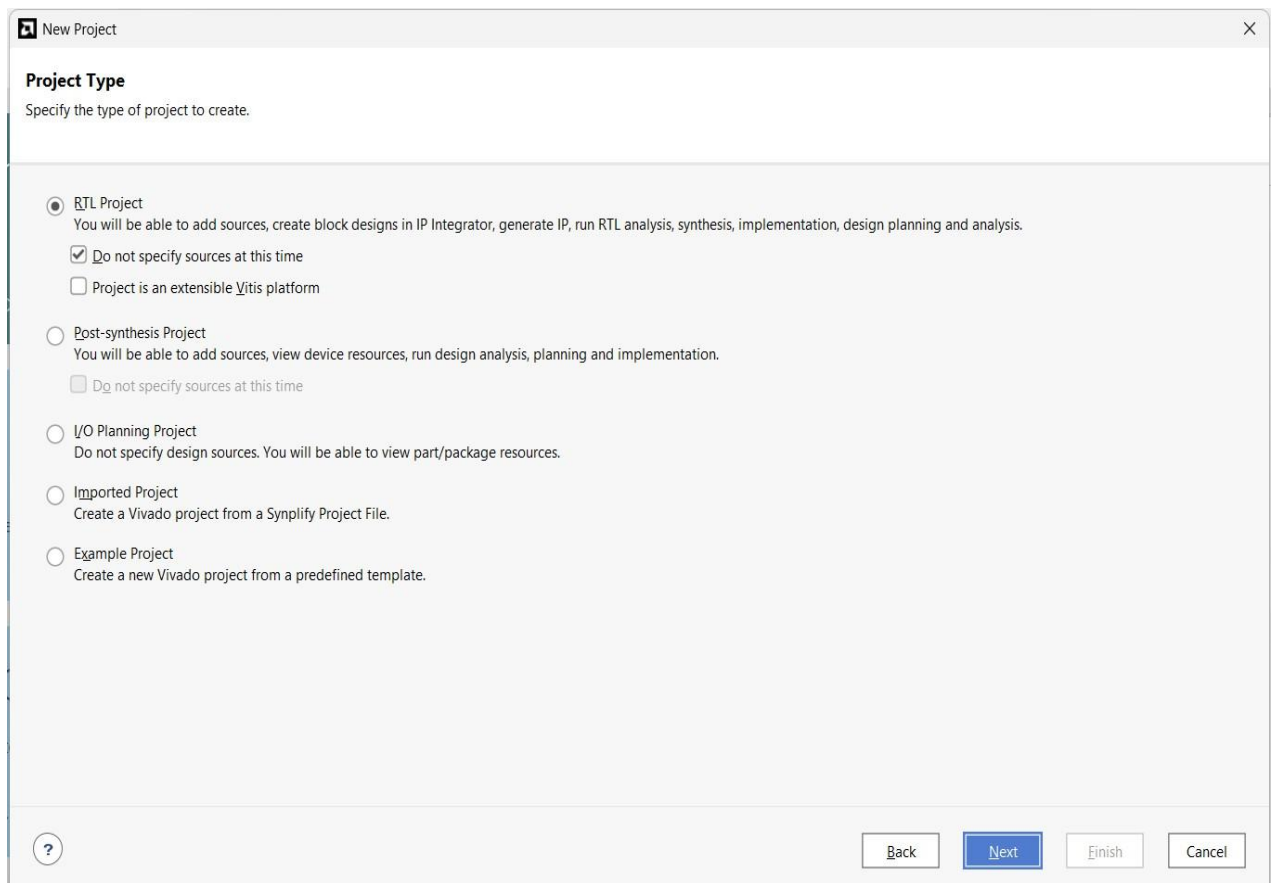
Project Name

Enter a name for your project and specify a directory where the project data files will be stored.



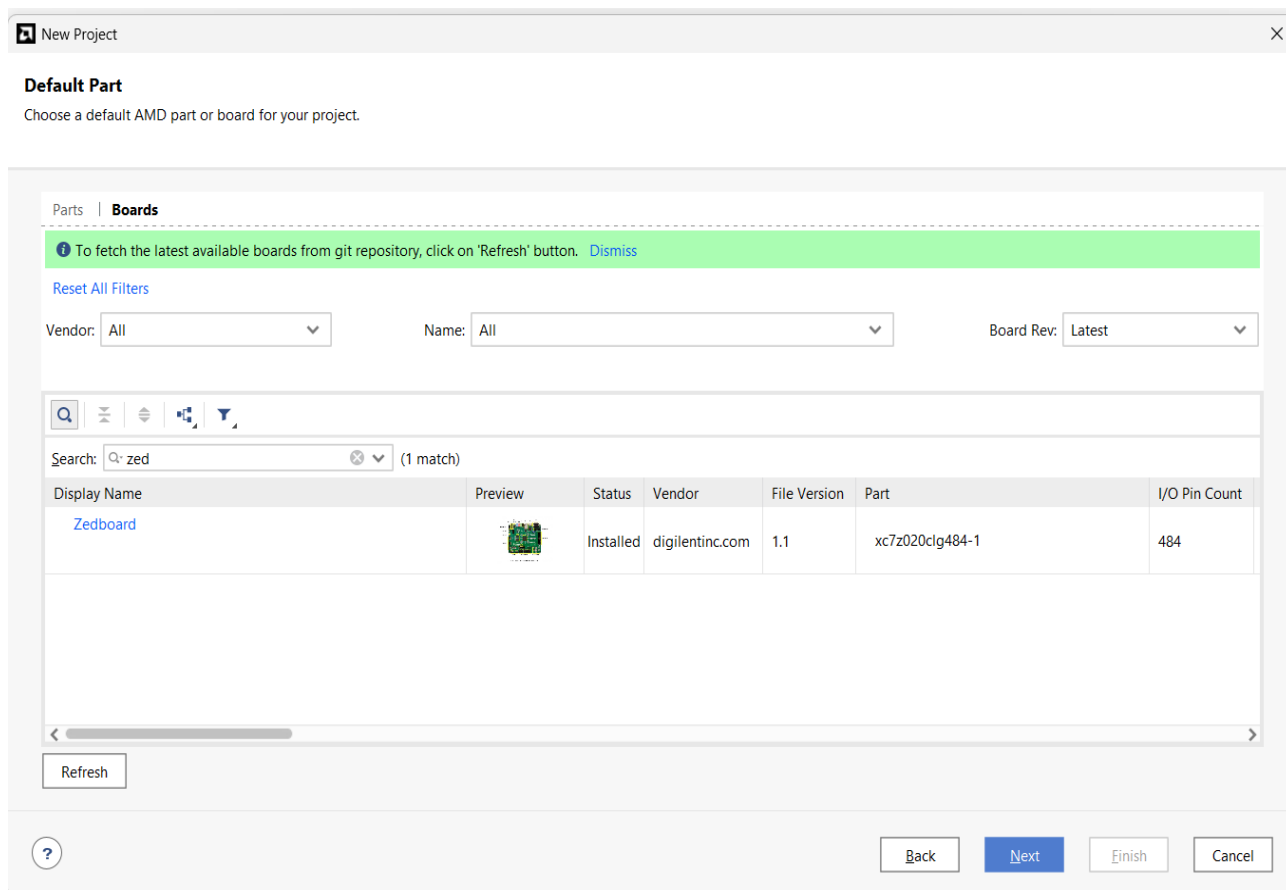
The screenshot shows the 'Project Name' dialog box. It has two text input fields: 'Project name:' with the value 'hanuman_thunai' and 'Project location:' with the value '/home/sathish'. Below these fields is a checked checkbox labeled 'Create project subdirectory'. Underneath the checkbox, it says 'Project will be created at: /home/sathish/hanuman_thunai'. At the bottom of the dialog, there are four buttons: a help button (question mark in a circle), 'Back', 'Next' (highlighted in blue), 'Finish', and 'Cancel'.

2. In the Project Type window, select **RTL Project** and check the "Do not specify sources at this time" checkbox. Click Next.

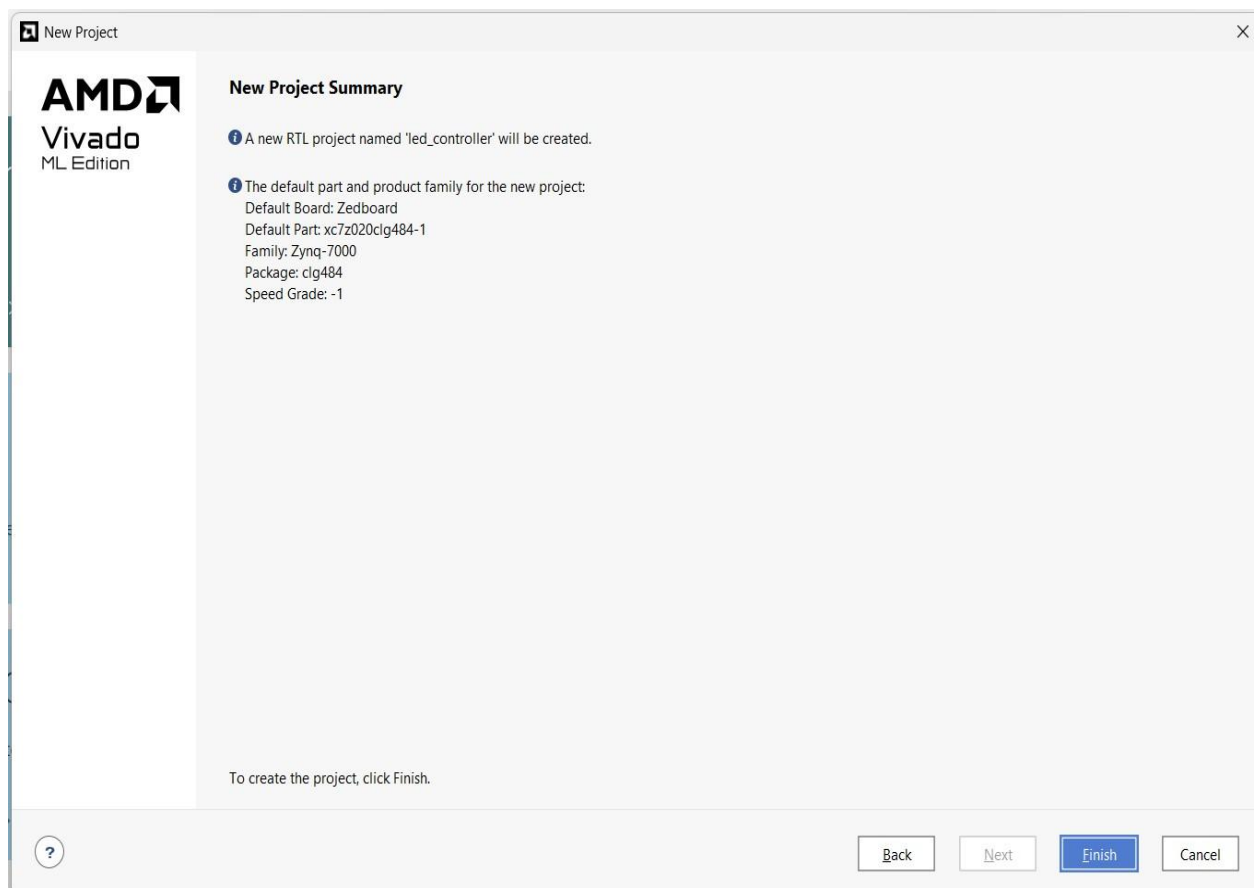


The screenshot shows the 'Project Type' dialog box. The title bar says 'New Project'. The main heading is 'Project Type' with the instruction 'Specify the type of project to create.' There are five radio button options: 'RTL Project' (selected), 'Post-synthesis Project', 'I/O Planning Project', 'Imported Project', and 'Example Project'. Each option has a description. Under 'RTL Project', there are two checkboxes: 'Do not specify sources at this time' (checked) and 'Project is an extensible Vitis platform' (unchecked). At the bottom of the dialog, there are four buttons: a help button (question mark in a circle), 'Back', 'Next' (highlighted in blue), 'Finish', and 'Cancel'.

3. In the Default Part window, switch to the **Boards** tab. Select the **ZedBoard Zynq Evaluation and Development Kit** from the list and click Next.

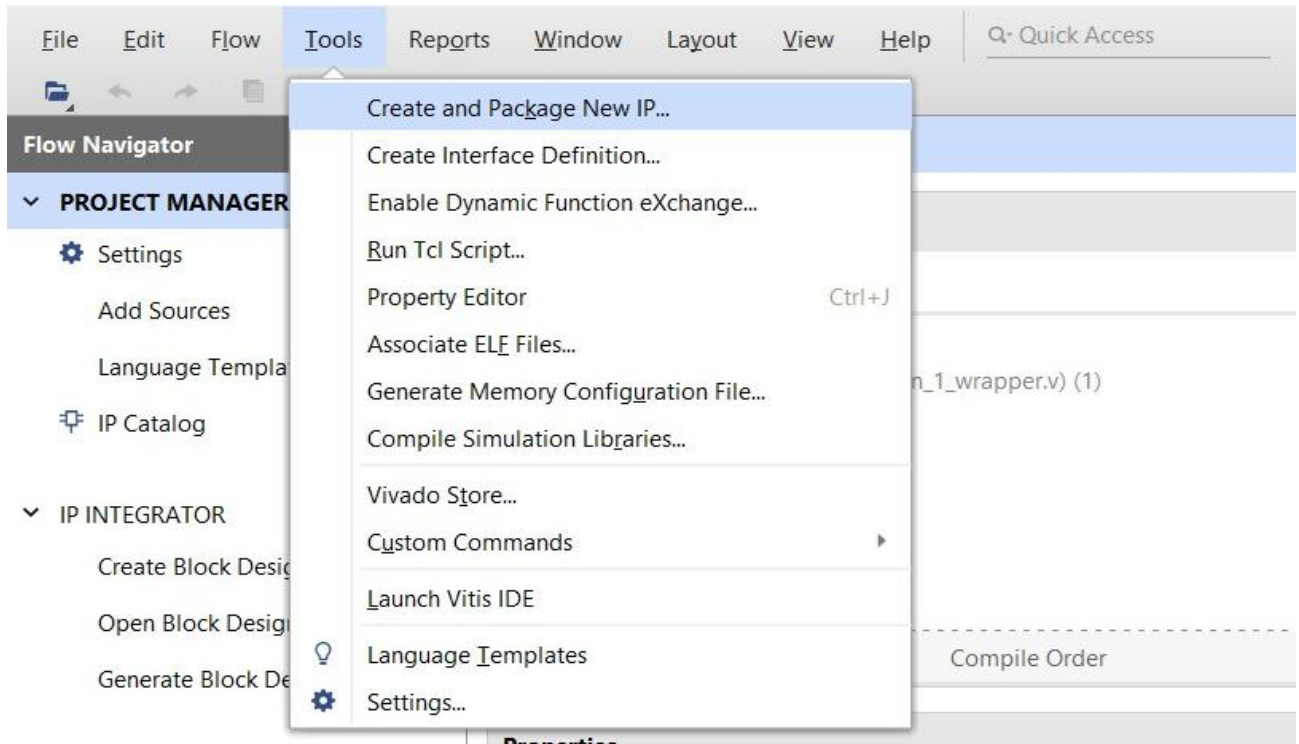


4. The New Project Summary window appears. Review the settings and click **Finish**. Vivado successfully builds the project and the new project window opens.

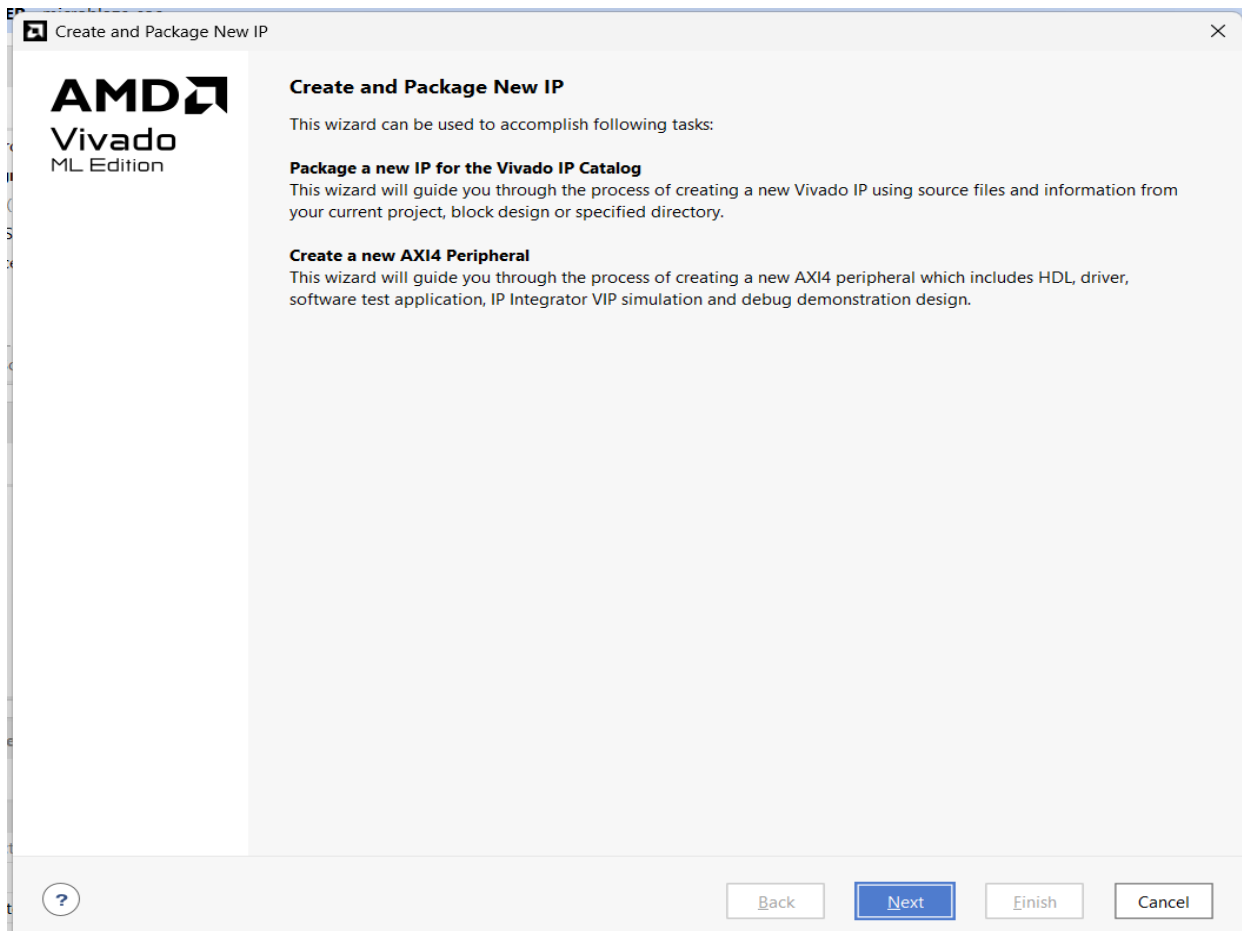


Create the Custom LED IP

1. With the base Vivado project created, from the menu select **Tools** → **Create and Package New IP**.



2. The "Create and Package New IP" window appears. Click "Next".



- On the next page, select **"Create a new AXI4 peripheral"** and click "Next".

Create and Package New IP

Create Peripheral, Package IP or Package a Block Design

Please select one of the following tasks.

Packaging Options

- ☐ Package your current project
Use the project as the source for creating a new IP Definition.
- ☐ Package a block design from the current project
Choose a block design as the source for creating a new IP Definition.
Select a block design: design_1
- ☐ Package a specified directory
Choose a directory as the source for creating a new IP Definition.

Create AXI4 Peripheral

- ☒ Create a new AXI4 peripheral
Create an AXI4 IP, driver, software test application, IP Integrator AXI4 VIP simulation and debug demonstration design.

Back Next Finish Cancel

- Now define the IP package name as **my_pid_controller_0**. A description and location can also be defined if needed. Click "Next".

Peripheral Details

Specify name, version and description for the new peripheral

Peripheral Details

Specify name, version and description for the new peripheral

Name: my_pid_controller_0

Version: 1.0

Display name: my_pid_controller_0_v1.0

Description: My new AXI IP

IP location: /home/sathish/ip_repo

☐ Overwrite existing

Back Next Finish Cancel

5. On the next page, the AXI bus interface is configured. The interface type used is **AXI Lite**, mode is **Slave**, and **4 registers of 32-bit data width** are used (slv_reg0 through slv_reg3). Click "Next".

Create and Package New IP

Add Interfaces
Add AXI4 interfaces supported by your peripheral

☐ Enable Interrupt Support

Interfaces
+ -
S00_AXI

Name: S00_AXI

Interface Type: Lite

Interface Mode: Slave

Data Width (Bits): 32

Memory Size (Bytes): 64

Number of Registers: 4 [4..512]

custom_led_ip_1_v1.0

Buttons: Back, Next, Finish, Cancel

6. On the final page, select **"Edit IP"** and click "Finish". A new Vivado window opens for editing the IP.

Create and Package New IP

AMD Vivado ML Edition

Create Peripheral

Peripheral Generation Summary

- 1. IP (xilinx.com:user:custom_led_ip_1:1.0) with 1 interface(s)
- 2. Driver(v1_00_a) and testapp [more info](#)
- 3. AXI4 VIP Simulation demonstration design [more info](#)
- 4. AXI4 Debug Hardware Simulation demonstration design [more info](#)

Peripheral created will be available in the catalog :
C:/SoC_project/microblaze_soc/./ip_repo

Next Steps:

- ☐ Add IP to the repository
- ☒ Edit IP
- ☐ Verify Peripheral IP using AXI4 VIP
- ☐ Verify peripheral IP using JTAG interface

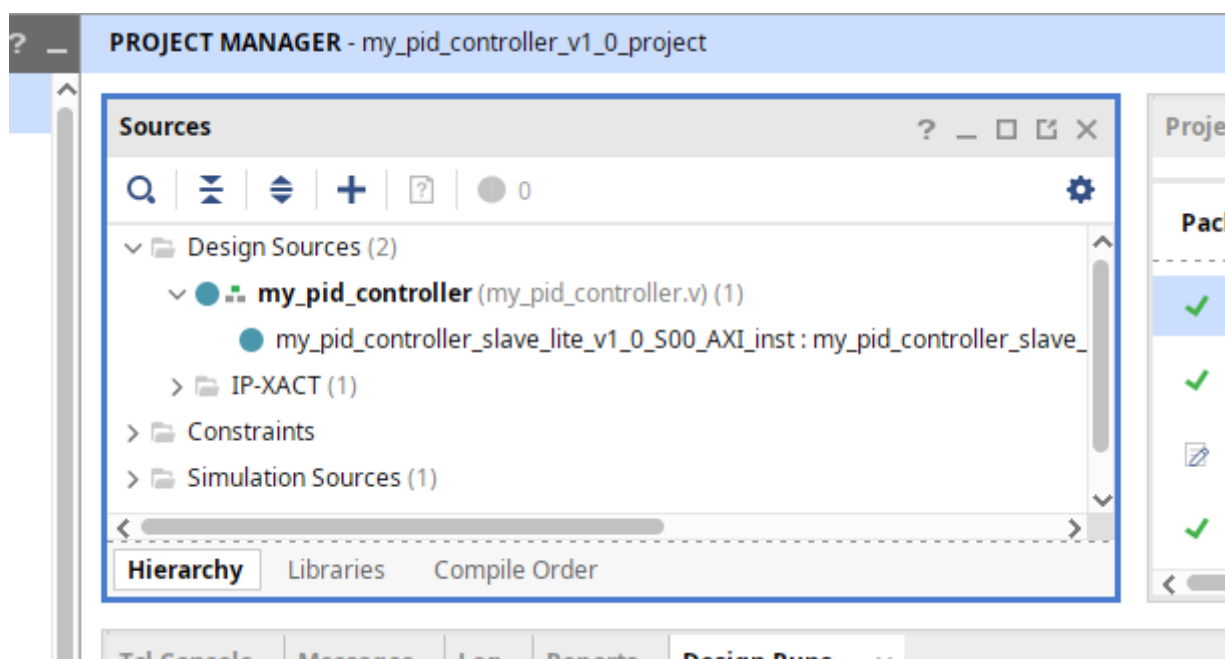
Buttons: Back, Next, Finish, Cancel

7. The package is initialized and the IP editor window opens with the generated AXI Lite slave template, containing 4 x 32-bit read/write registers. The generated IP is a basic template where register values can be read and written over the AXI bus. In this project, we add custom LED controller logic to this template — the hardware will read the mode from `slv_reg0`, the LED data from `slv_reg1`, and the speed setting from `slv_reg2`. Once the IP is created and packaged, it can be reused across any board by simply changing the project settings and constraints.

3. Add the Custom LED Logic to the Peripheral

The following steps are done in the IP editor window (not the base project window).

1. The custom IP package with the name **my_pid_controller** appears in the IP editor. The custom LED logic is to be added to the **my_pid_controller_slave_lite_v1_0_S00_AXI.v** module.



2. In the Sources tree, open the file **my_pid_controller_slave_lite_v1_0_S00_AXI.v** (the top module).

```
`timescale 1 ns / 1 ps
```

```
module my_pid_controller_slave_lite_v1_0_S00_AXI #  
(  
    // Users to add parameters here  
  
    // User parameters ends  
    // Do not modify the parameters beyond this line  
  
    // Width of S_AXI data bus  
    parameter integer C_S_AXI_DATA_WIDTH = 32,  
    // Width of S_AXI address bus  
    parameter integer C_S_AXI_ADDR_WIDTH = 5  
)  
(  
    // Users to add ports here
```

```

// User ports ends
// Do not modify the ports beyond this line

// Global Clock Signal
input wire S_AXI_ACLK,
// Global Reset Signal. This Signal is Active LOW
input wire S_AXI_ARESETN,
// Write address (issued by master, accepted by Slave)
input wire [C_S_AXI_ADDR_WIDTH-1 : 0] S_AXI_AWADDR,
// Write channel Protection type. This signal indicates the
// privilege and security level of the transaction, and whether
// the transaction is a data access or an instruction access.
input wire [2 : 0] S_AXI_AWPROT,
// Write address valid. This signal indicates that the master signaling
// valid write address and control information.
input wire S_AXI_AWVALID,
// Write address ready. This signal indicates that the slave is ready
// to accept an address and associated control signals.
output wire S_AXI_AWREADY,
// Write data (issued by master, accepted by Slave)
input wire [C_S_AXI_DATA_WIDTH-1 : 0] S_AXI_WDATA,
// Write strobes. This signal indicates which byte lanes hold
// valid data. There is one write strobe bit for each eight
// bits of the write data bus.
input wire [(C_S_AXI_DATA_WIDTH/8)-1 : 0] S_AXI_WSTRB,
// Write valid. This signal indicates that valid write
// data and strobes are available.
input wire S_AXI_WVALID,
// Write ready. This signal indicates that the slave
// can accept the write data.
output wire S_AXI_WREADY,
// Write response. This signal indicates the status
// of the write transaction.
output wire [1 : 0] S_AXI_BRESP,
// Write response valid. This signal indicates that the channel
// is signaling a valid write response.
output wire S_AXI_BVALID,
// Response ready. This signal indicates that the master
// can accept a write response.
input wire S_AXI_BREADY,
// Read address (issued by master, accepted by Slave)
input wire [C_S_AXI_ADDR_WIDTH-1 : 0] S_AXI_ARADDR,
// Protection type. This signal indicates the privilege
// and security level of the transaction, and whether the
// transaction is a data access or an instruction access.
input wire [2 : 0] S_AXI_ARPROT,
// Read address valid. This signal indicates that the channel
// is signaling valid read address and control information.

```

```

input wire S_AXI_ARVALID,
// Read address ready. This signal indicates that the slave is
// ready to accept an address and associated control signals.
output wire S_AXI_ARREADY,
// Read data (issued by slave)
output wire [C_S_AXI_DATA_WIDTH-1 : 0] S_AXI_RDATA,
// Read response. This signal indicates the status of the
// read transfer.
output wire [1 : 0] S_AXI_RRESP,
// Read valid. This signal indicates that the channel is
// signaling the required read data.
output wire S_AXI_RVALID,
// Read ready. This signal indicates that the master can
// accept the read data and response information.
input wire S_AXI_RREADY
);

// AXI4LITE signals
reg [C_S_AXI_ADDR_WIDTH-1 : 0] axi_awaddr;
reg axi_awready;
reg axi_wready;
reg [1 : 0] axi_bresp;
reg axi_bvalid;
reg [C_S_AXI_ADDR_WIDTH-1 : 0] axi_araddr;
reg axi_arready;
reg [1 : 0] axi_rresp;
reg axi_rvalid;

// Example-specific design signals
localparam integer ADDR_LSB = (C_S_AXI_DATA_WIDTH/32) + 1;
localparam integer OPT_MEM_ADDR_BITS = 2;

//-- Number of Slave Registers 8
reg [C_S_AXI_DATA_WIDTH-1:0] slv_reg0;
reg [C_S_AXI_DATA_WIDTH-1:0] slv_reg1;
reg [C_S_AXI_DATA_WIDTH-1:0] slv_reg2;
reg [C_S_AXI_DATA_WIDTH-1:0] slv_reg3;
reg [C_S_AXI_DATA_WIDTH-1:0] slv_reg4;
reg [C_S_AXI_DATA_WIDTH-1:0] slv_reg5;
reg [C_S_AXI_DATA_WIDTH-1:0] slv_reg6;
reg [C_S_AXI_DATA_WIDTH-1:0] slv_reg7;
integer byte_index;

// I/O Connections assignments
assign S_AXI_AWREADY = axi_awready;
assign S_AXI_WREADY = axi_wready;
assign S_AXI_BRESP = axi_bresp;
assign S_AXI_BVALID = axi_bvalid;

```

```

assign S_AXI_ARREADY = axi_arready;
assign S_AXI_RRESP    = axi_rresp;
assign S_AXI_RVALID   = axi_rvalid;

//state machine variables
reg [1:0] state_write;
reg [1:0] state_read;
//State machine local parameters
localparam Idle = 2'b00, Raddr = 2'b10, Rdata = 2'b11, Waddr = 2'b10, Wdata = 2'b11;

// Implement Write state machine
always @(posedge S_AXI_ACLK)
begin
    if (S_AXI_ARESETN == 1'b0)
    begin
        axi_awready <= 0;
        axi_wready <= 0;
        axi_bvalid <= 0;
        axi_bresp <= 0;
        axi_awaddr <= 0;
        state_write <= Idle;
    end
    else
    begin
        case(state_write)
            Idle:
            begin
                if(S_AXI_ARESETN == 1'b1)
                begin
                    axi_awready <= 1'b1;
                    axi_wready <= 1'b1;
                    state_write <= Waddr;
                end
                else state_write <= state_write;
            end
            Waddr:
            begin
                if (S_AXI_AWVALID && S_AXI_AWREADY)
                begin
                    axi_awaddr <= S_AXI_AWADDR;
                    if(S_AXI_WVALID)
                    begin
                        axi_awready <= 1'b1;
                        state_write <= Waddr;
                        axi_bvalid <= 1'b1;
                    end
                    else
                    begin

```



```

        axi_awready <= 1'b0;
        state_write <= Wdata;
        if (S_AXI_BREADY && axi_bvalid) axi_bvalid <= 1'b0;
    end
end
else
begin
    state_write <= state_write;
    if (S_AXI_BREADY && axi_bvalid) axi_bvalid <= 1'b0;
end
end
Wdata:
begin
    if (S_AXI_WVALID)
    begin
        state_write <= Waddr;
        axi_bvalid <= 1'b1;
        axi_awready <= 1'b1;
    end
    else
    begin
        state_write <= state_write;
        if (S_AXI_BREADY && axi_bvalid) axi_bvalid <= 1'b0;
    end
end
endcase
end
end

```

// Implement memory mapped register select and write logic generation

always @(posedge S_AXI_ACLK)

begin

if (S_AXI_ARESETN == 1'b0)

begin

slv_reg0 <= 0;

slv_reg1 <= 0;

slv_reg2 <= 0;

slv_reg3 <= 0;

slv_reg4 <= 0;

slv_reg5 <= 0;

slv_reg6 <= 0;

slv_reg7 <= 0;

end

else begin

if (S_AXI_WVALID)

begin

case ((S_AXI_AWVALID) ? S_AXI_AWADDR[ADDR_LSB+OPT_MEM_ADDR_BITS:ADDR_LSB]
: axi_awaddr[ADDR_LSB+OPT_MEM_ADDR_BITS:ADDR_LSB])

```

3'h0:
  for ( byte_index = 0; byte_index <= (C_S_AXI_DATA_WIDTH/8)-1; byte_index =
byte_index+1 )
    if ( S_AXI_WSTRB[byte_index] == 1 ) begin
      slv_reg0[(byte_index*8) +: 8] <= S_AXI_WDATA[(byte_index*8) +: 8];
    end
3'h1:
  for ( byte_index = 0; byte_index <= (C_S_AXI_DATA_WIDTH/8)-1; byte_index =
byte_index+1 )
    if ( S_AXI_WSTRB[byte_index] == 1 ) begin
      slv_reg1[(byte_index*8) +: 8] <= S_AXI_WDATA[(byte_index*8) +: 8];
    end
3'h2:
  for ( byte_index = 0; byte_index <= (C_S_AXI_DATA_WIDTH/8)-1; byte_index =
byte_index+1 )
    if ( S_AXI_WSTRB[byte_index] == 1 ) begin
      slv_reg2[(byte_index*8) +: 8] <= S_AXI_WDATA[(byte_index*8) +: 8];
    end
3'h3:
  for ( byte_index = 0; byte_index <= (C_S_AXI_DATA_WIDTH/8)-1; byte_index =
byte_index+1 )
    if ( S_AXI_WSTRB[byte_index] == 1 ) begin
      slv_reg3[(byte_index*8) +: 8] <= S_AXI_WDATA[(byte_index*8) +: 8];
    end
3'h4:
  for ( byte_index = 0; byte_index <= (C_S_AXI_DATA_WIDTH/8)-1; byte_index =
byte_index+1 )
    if ( S_AXI_WSTRB[byte_index] == 1 ) begin
      slv_reg4[(byte_index*8) +: 8] <= S_AXI_WDATA[(byte_index*8) +: 8];
    end
3'h5:
  for ( byte_index = 0; byte_index <= (C_S_AXI_DATA_WIDTH/8)-1; byte_index =
byte_index+1 )
    if ( S_AXI_WSTRB[byte_index] == 1 ) begin
      slv_reg5[(byte_index*8) +: 8] <= S_AXI_WDATA[(byte_index*8) +: 8];
    end
3'h6:
  for ( byte_index = 0; byte_index <= (C_S_AXI_DATA_WIDTH/8)-1; byte_index =
byte_index+1 )
    if ( S_AXI_WSTRB[byte_index] == 1 ) begin
      slv_reg6[(byte_index*8) +: 8] <= S_AXI_WDATA[(byte_index*8) +: 8];
    end
3'h7:
  for ( byte_index = 0; byte_index <= (C_S_AXI_DATA_WIDTH/8)-1; byte_index =
byte_index+1 )
    if ( S_AXI_WSTRB[byte_index] == 1 ) begin
      slv_reg7[(byte_index*8) +: 8] <= S_AXI_WDATA[(byte_index*8) +: 8];
    end

```

```

default : begin
    slv_reg0 <= slv_reg0;
    slv_reg1 <= slv_reg1;
    slv_reg2 <= slv_reg2;
    slv_reg3 <= slv_reg3;
    slv_reg4 <= slv_reg4;
    slv_reg5 <= slv_reg5;
    slv_reg6 <= slv_reg6;
    slv_reg7 <= slv_reg7;
end

```

```

endcase
end

```

```

end

```

```

// Implement read state machine

```

```

always @(posedge S_AXI_ACLK)

```

```

begin

```

```

    if (S_AXI_ARESETN == 1'b0)

```

```

    begin

```

```

        axi_arready <= 1'b0;

```

```

        axi_rvalid <= 1'b0;

```

```

        axi_rresp <= 1'b0;

```

```

        state_read <= Idle;

```

```

    end

```

```

    else

```

```

    begin

```

```

        case(state_read)

```

```

            Idle:

```

```

            begin

```

```

                if (S_AXI_ARESETN == 1'b1)

```

```

                begin

```

```

                    state_read <= Raddr;

```

```

                    axi_arready <= 1'b1;

```

```

                end

```

```

                else state_read <= state_read;

```

```

            end

```

```

            Raddr:

```

```

            begin

```

```

                if (S_AXI_ARVALID && S_AXI_ARREADY)

```

```

                begin

```

```

                    state_read <= Rdata;

```

```

                    axi_araddr <= S_AXI_ARADDR;

```

```

                    axi_rvalid <= 1'b1;

```

```

                    axi_arready <= 1'b0;

```

```

                end

```

```

                else state_read <= state_read;

```

```

            end

```

```

Rdata:
begin
    if (S_AXI_RVALID && S_AXI_RREADY)
    begin
        axi_rvalid <= 1'b0;
        axi_arready <= 1'b1;
        state_read <= Raddr;
    end
    else state_read <= state_read;
end
endcase
end
end

// -----
// IMPLEMENT MEMORY MAPPED REGISTER SELECT & READ LOGIC
// -----
// We declare the control_effort wire *before* the AXI Read Assignment
// so the compiler knows it exists when intercepting slv_reg6.
wire signed [31:0] control_effort;

assign S_AXI_RDATA = (axi_araddr[ADDR_LSB+OPT_MEM_ADDR_BITS:ADDR_LSB] == 3'h0) ?
slv_reg0 :
    (axi_araddr[ADDR_LSB+OPT_MEM_ADDR_BITS:ADDR_LSB] == 3'h1) ? slv_reg1 :
    (axi_araddr[ADDR_LSB+OPT_MEM_ADDR_BITS:ADDR_LSB] == 3'h2) ? slv_reg2 :
    (axi_araddr[ADDR_LSB+OPT_MEM_ADDR_BITS:ADDR_LSB] == 3'h3) ? slv_reg3 :
    (axi_araddr[ADDR_LSB+OPT_MEM_ADDR_BITS:ADDR_LSB] == 3'h4) ? slv_reg4 :
    (axi_araddr[ADDR_LSB+OPT_MEM_ADDR_BITS:ADDR_LSB] == 3'h5) ? slv_reg5 :
    (axi_araddr[ADDR_LSB+OPT_MEM_ADDR_BITS:ADDR_LSB] == 3'h6) ?
control_effort : // CRITICAL: Reads from your PID Logic instead of slv_reg6!
    (axi_araddr[ADDR_LSB+OPT_MEM_ADDR_BITS:ADDR_LSB] == 3'h7) ? slv_reg7 :
0;

// -----
// USER LOGIC
// -----
wire signed [31:0] kp, ki, kd;
wire signed [31:0] error, integral, derivative;
wire signed [31:0] p_term, i_term, d_term;

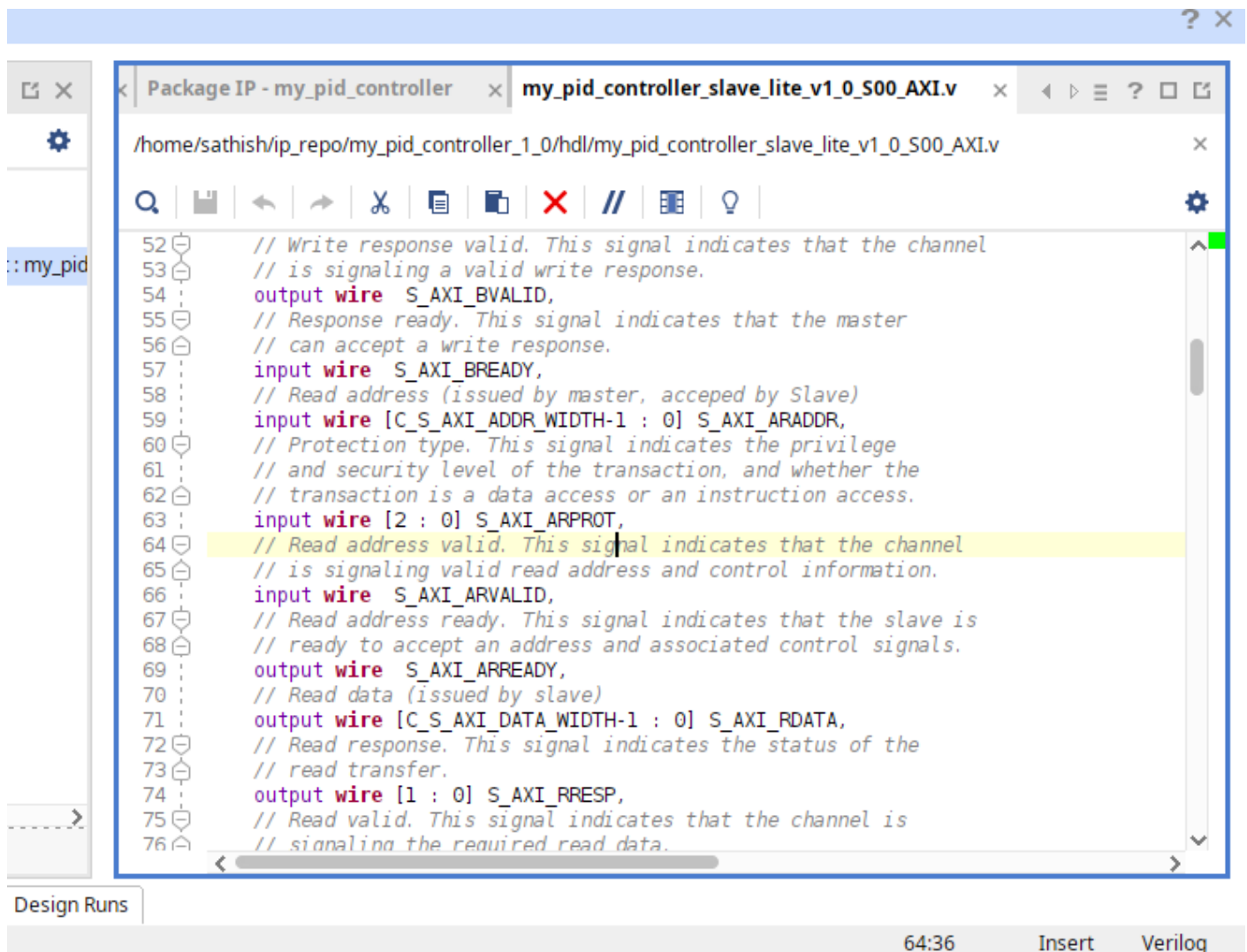
assign kp = slv_reg0;
assign ki = slv_reg1;
assign kd = slv_reg2;
assign error = slv_reg3;
assign integral = slv_reg4;
assign derivative = slv_reg5;

```

```
// Hardware parallel multiplication
assign p_term = kp * error;
assign i_term = ki * integral;
assign d_term = kd * derivative;

// Hardware accumulation
assign control_effort = p_term + i_term + d_term;
// User logic ends

endmodule
```



3. Save the file. The file is now integrated into the IP hierarchy with the full LED logic.
4. Click on the **"Package IP"** tab and select **"File Groups"**.
5. Click the **"Merge changes from IP File Group Wizard"** link. The IP File Groups entry should now show a tick mark.

6. Now click on **"Review and Package IP"** and then click **"Re-Package IP"**. A "Close Project" dialog appears — click "Yes". The IP is now packaged and ready to use.

4. Add the IP to the Block Design

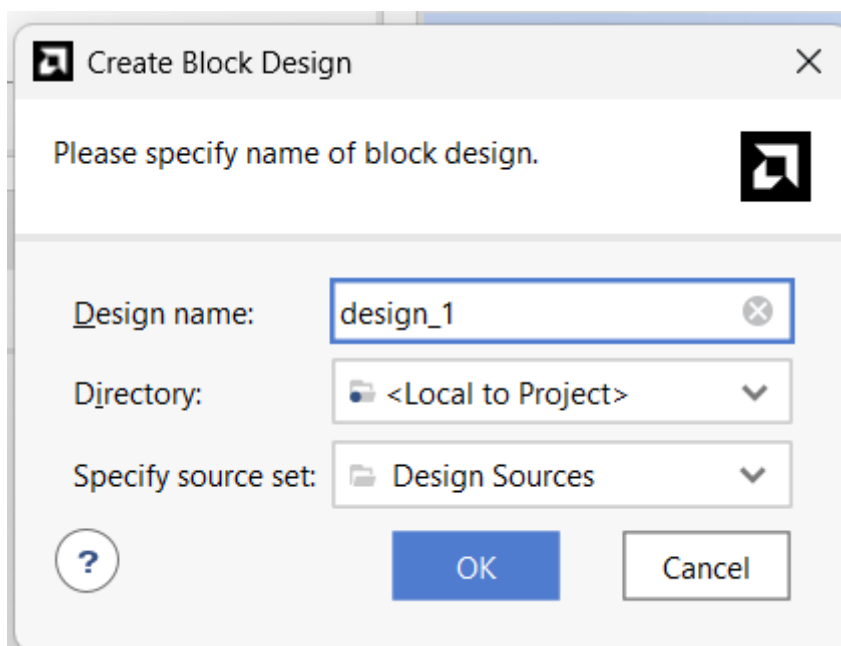
1. Back in the main Vivado project window, go to the Flow Navigator and click **"Create Block Design"** under IP Integrator. Name it ``design_1``.

IP INTEGRATOR

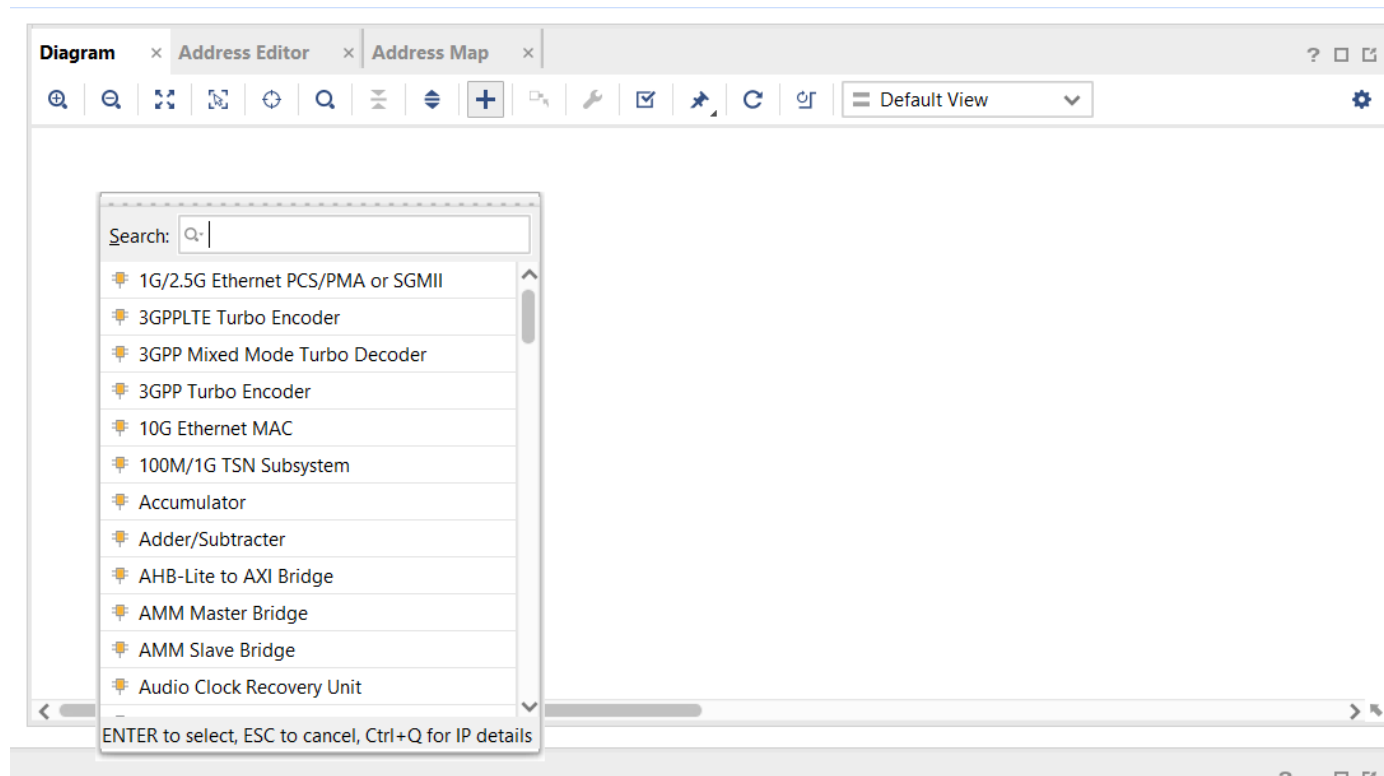
[Create Block Design](#)

[Open Block Design](#)

[Generate Block Design](#)



2. Click the **"Add IP"** icon (+) in the block design canvas.

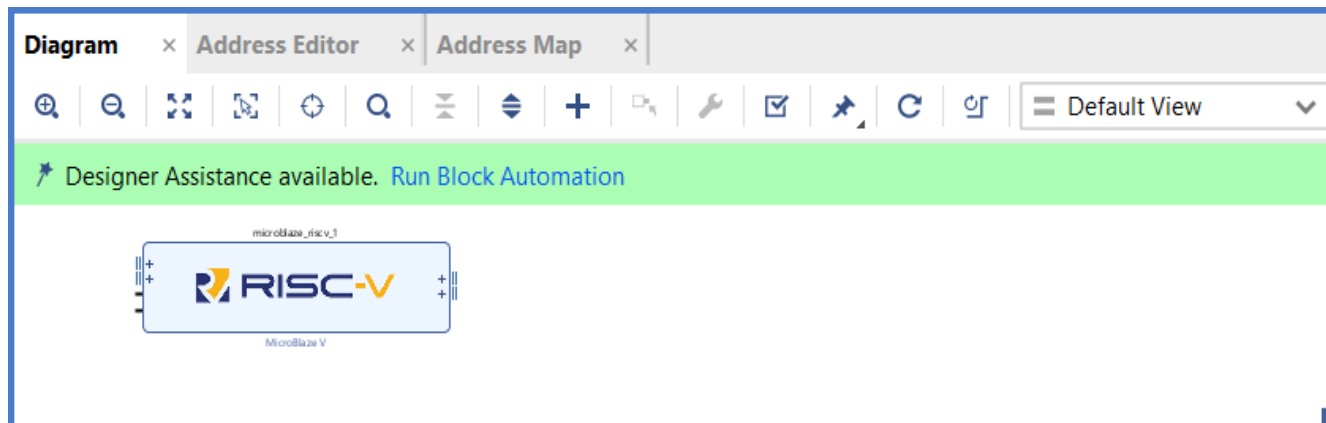


3. Search for and add the following IP blocks to the design:

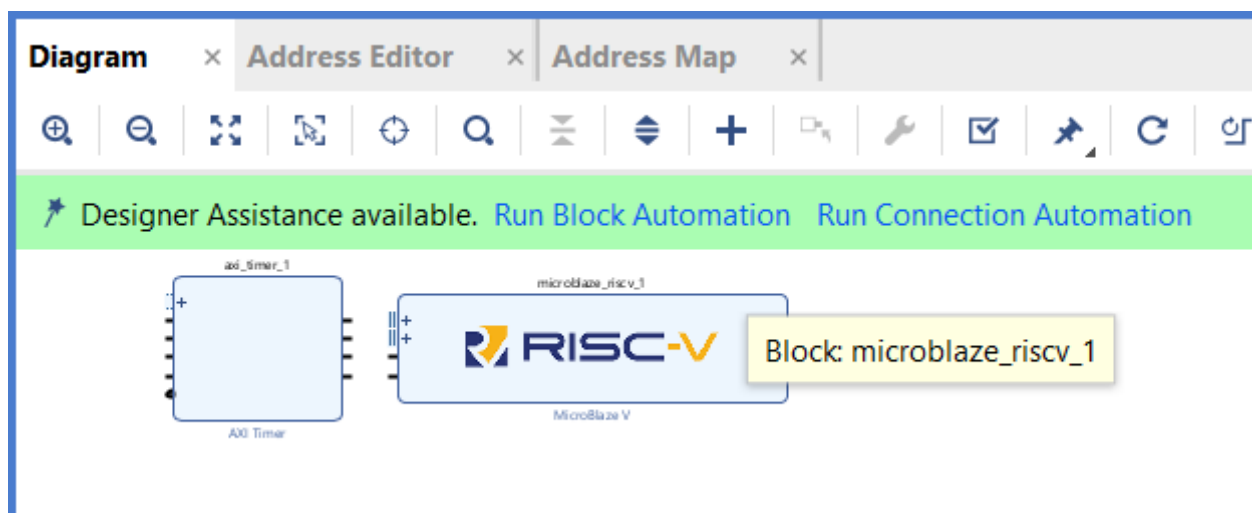
- **MicroBlaze-V**
- **AXI Timer**
- **my_pid_controller_0** (our custom IP)

4. The blocks appear in the block design and the message **"Designer Assistance available"** will appear at the top.

5. Click **"Run Block Automation"**. In the dialog, ensure local memory is set to 64 KB (BRAM). This is critical — the MicroBlaze-V needs this memory to store and run the C program. Click OK.

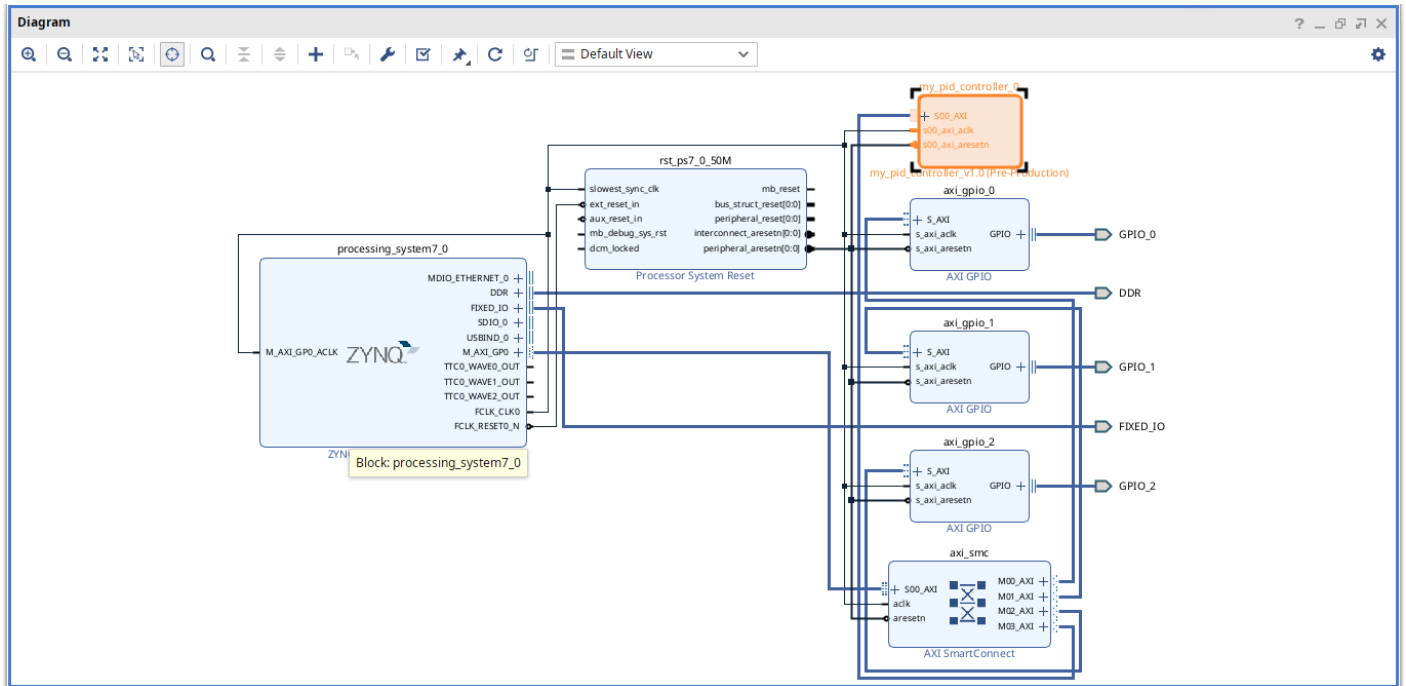


6. Click **"Run Connection Automation"**. Select **"All Automation"** and click OK. Vivado automatically creates the AXI Interconnect and connects all clocks, resets, and AXI interfaces between MicroBlaze - V and the three peripherals.

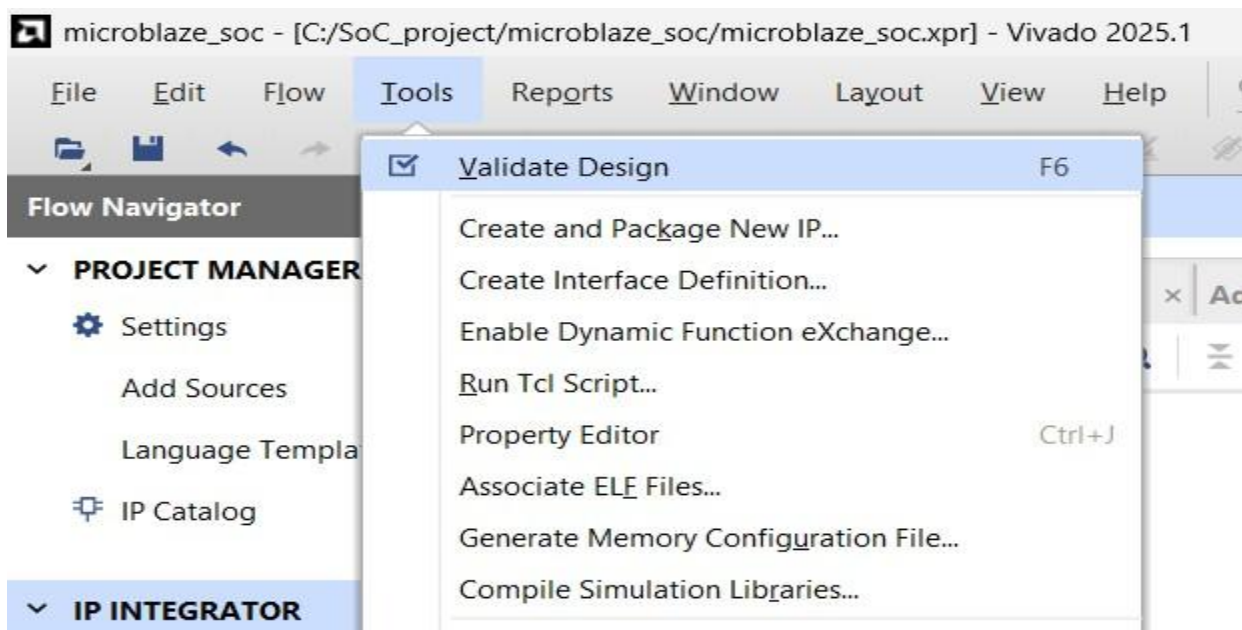


7. Right-click the ****LED_OUT**** port on the custom LED IP block and select ****"Make External"****. This creates a top-level output port that will connect to the physical FPGA pins. The UART port (uart_rtl_0) will already be external automatically.

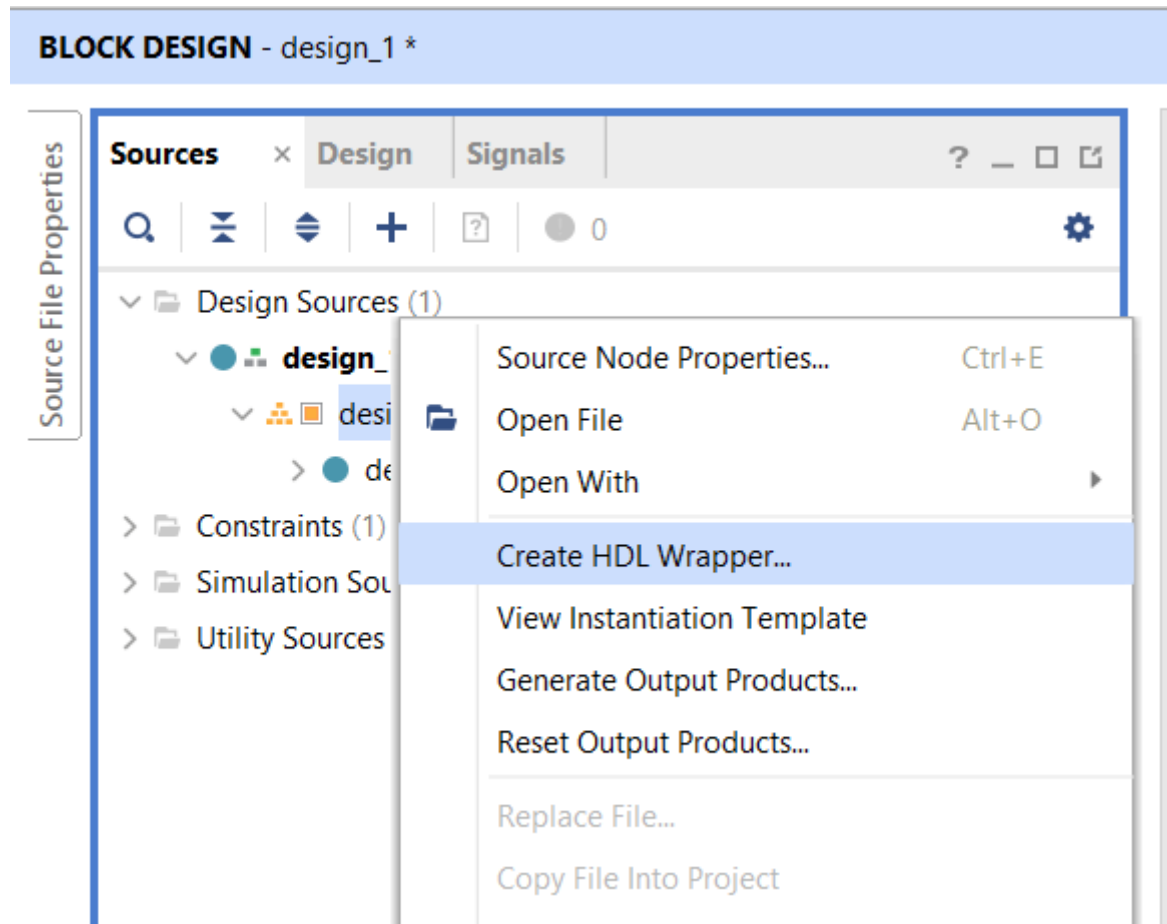
8. The block design also automatically generates a ****clk_100MHz**** external clock input and a ****reset_rtl_0**** external reset input.



9. From the Block Design toolbar, click **"Validate Design"**. A popup confirms the design is valid with no errors. Click OK.



10. Click **"Generate Block Design"** under IP Integrator, then click **"Generate"** on the output products popup.
11. Right-click the block design in the Sources panel and select **"Create HDL Wrapper"**. Vivado automatically generates the top-level Verilog wrapper for the entire block design.



12. After the wrapper is created, it appears as the top-level module in the Sources hierarchy.

13. Click **"Generate Bitstream"** under Program and Debug in the Flow Navigator. This compiles the entire design into FPGA configuration bits. After completion, click "Open Implemented Design" to review timing, power, and DRC reports.

- SIMULATION
 - Run Simulation
 - RTL ANALYSIS
 - Run Linter
 - Open Elaborated Design
 - SYNTHESIS
 - Run Synthesis
 - Open Synthesized Design
 - IMPLEMENTATION
 - Run Implementation
 - Open Implemented Design
 - PROGRAM AND DEBUG
 - Generate Bitstream
 - Open Hardware Manager

Pin Constraints (XDC File)

Before generating the bitstream, add a constraints file to map the block design ports to physical FPGA pins. In the Sources panel, right-click Constraints → Add Sources → Create a new file named `zedboard.xdc`. Add the following:

```
## --- CLOCK (100 MHz) ---
set_property PACKAGE_PIN Y9 [get_ports S_AXI_ACLK]
set_property IOSTANDARD LVCMOS33 [get_ports S_AXI_ACLK]
create_clock -period 10.000 -name sys_clk_pin -waveform {0.000 5.000} -add [get_ports S_AXI_ACLK]

## --- OLED INTERFACE (OLED_GPIO_BASE) ---
# SCLK
set_property PACKAGE_PIN AB12 [get_ports {oled_gpio_tri_io[0]}]
# SDIN
set_property PACKAGE_PIN AA12 [get_ports {oled_gpio_tri_io[1]}]
# RES
set_property PACKAGE_PIN U10 [get_ports {oled_gpio_tri_io[2]}]
# DC
set_property PACKAGE_PIN U9 [get_ports {oled_gpio_tri_io[3]}]
# VDD
set_property PACKAGE_PIN U11 [get_ports {oled_gpio_tri_io[4]}]
# VBAT
set_property PACKAGE_PIN U12 [get_ports {oled_gpio_tri_io[5]}]

set_property IOSTANDARD LVCMOS33 [get_ports -filter { NAME =~ "oled_gpio*" }]

## --- SWITCHES (SW_GPIO_BASE) - Target Speed Input ---
set_property PACKAGE_PIN F22 [get_ports {sw_gpio_tri_io[0]}]
set_property PACKAGE_PIN G22 [get_ports {sw_gpio_tri_io[1]}]
set_property PACKAGE_PIN H22 [get_ports {sw_gpio_tri_io[2]}]
set_property PACKAGE_PIN F21 [get_ports {sw_gpio_tri_io[3]}]
set_property PACKAGE_PIN H19 [get_ports {sw_gpio_tri_io[4]}]
set_property PACKAGE_PIN H18 [get_ports {sw_gpio_tri_io[5]}]
set_property PACKAGE_PIN H17 [get_ports {sw_gpio_tri_io[6]}]
set_property PACKAGE_PIN M15 [get_ports {sw_gpio_tri_io[7]}]

set_property IOSTANDARD LVCMOS33 [get_ports -filter { NAME =~ "sw_gpio*" }]

## --- PUSH BUTTONS (BTN_GPIO_BASE) - Controls ---
# BTN_RIGHT (Center/Manual Control)
set_property PACKAGE_PIN R18 [get_ports {btn_gpio_tri_io[0]}]
# BTN_LEFT
set_property PACKAGE_PIN N15 [get_ports {btn_gpio_tri_io[1]}]
# BTN_DOWN
set_property PACKAGE_PIN R16 [get_ports {btn_gpio_tri_io[2]}]
# BTN_UP
set_property PACKAGE_PIN T18 [get_ports {btn_gpio_tri_io[3]}]
# BTN_CENTER (Emergency Stop)
set_property PACKAGE_PIN P16 [get_ports {btn_gpio_tri_io[4]}]

set_property IOSTANDARD LVCMOS33 [get_ports -filter { NAME =~ "btn_gpio*" }]
```

```
## --- RESET ---
```

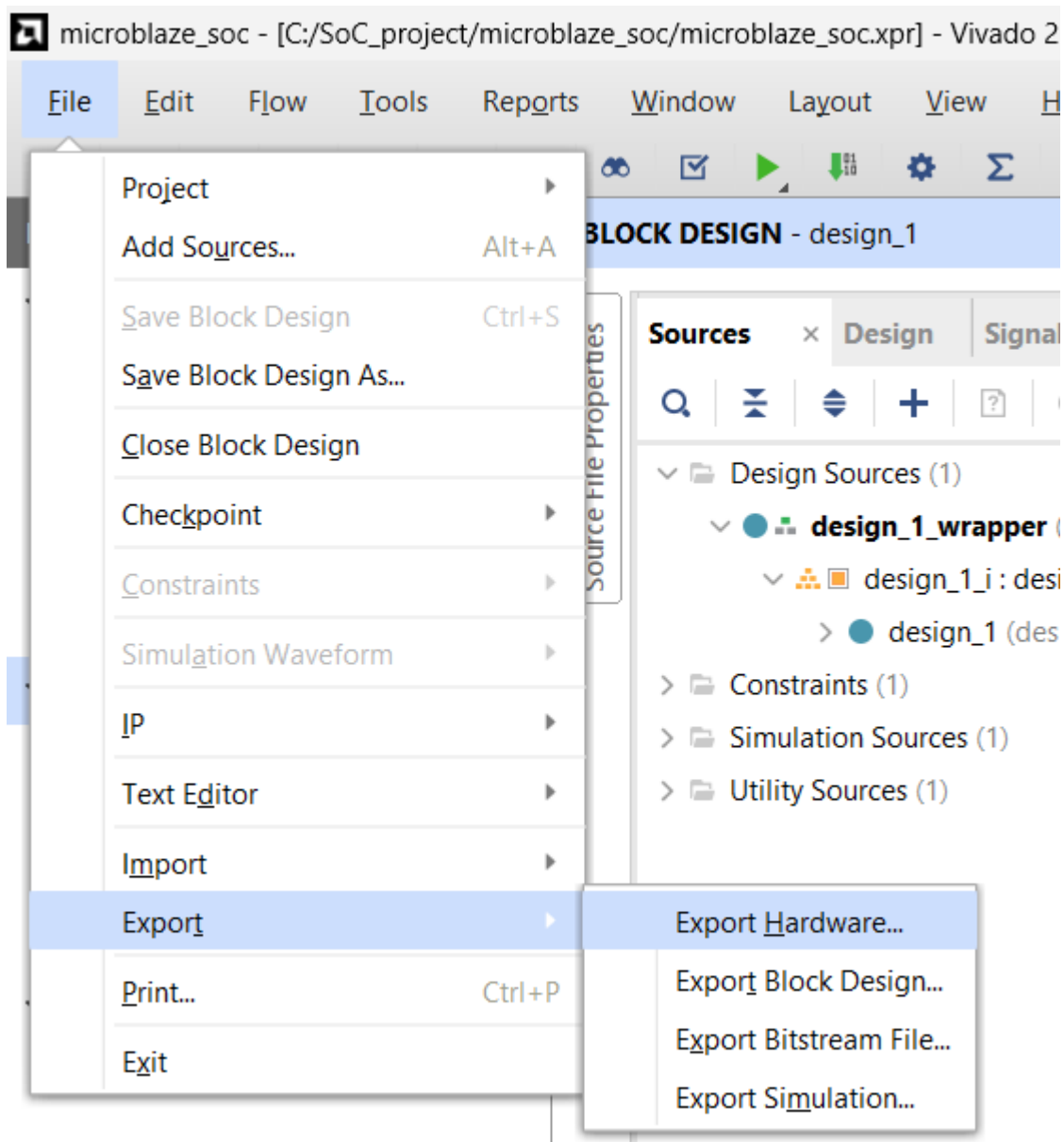
```
set_property PACKAGE_PIN P16 [get_ports S_AXI_ARESETN]
```

```
set_property IOSTANDARD LVCMOS33 [get_ports S_AXI_ARESETN]
```

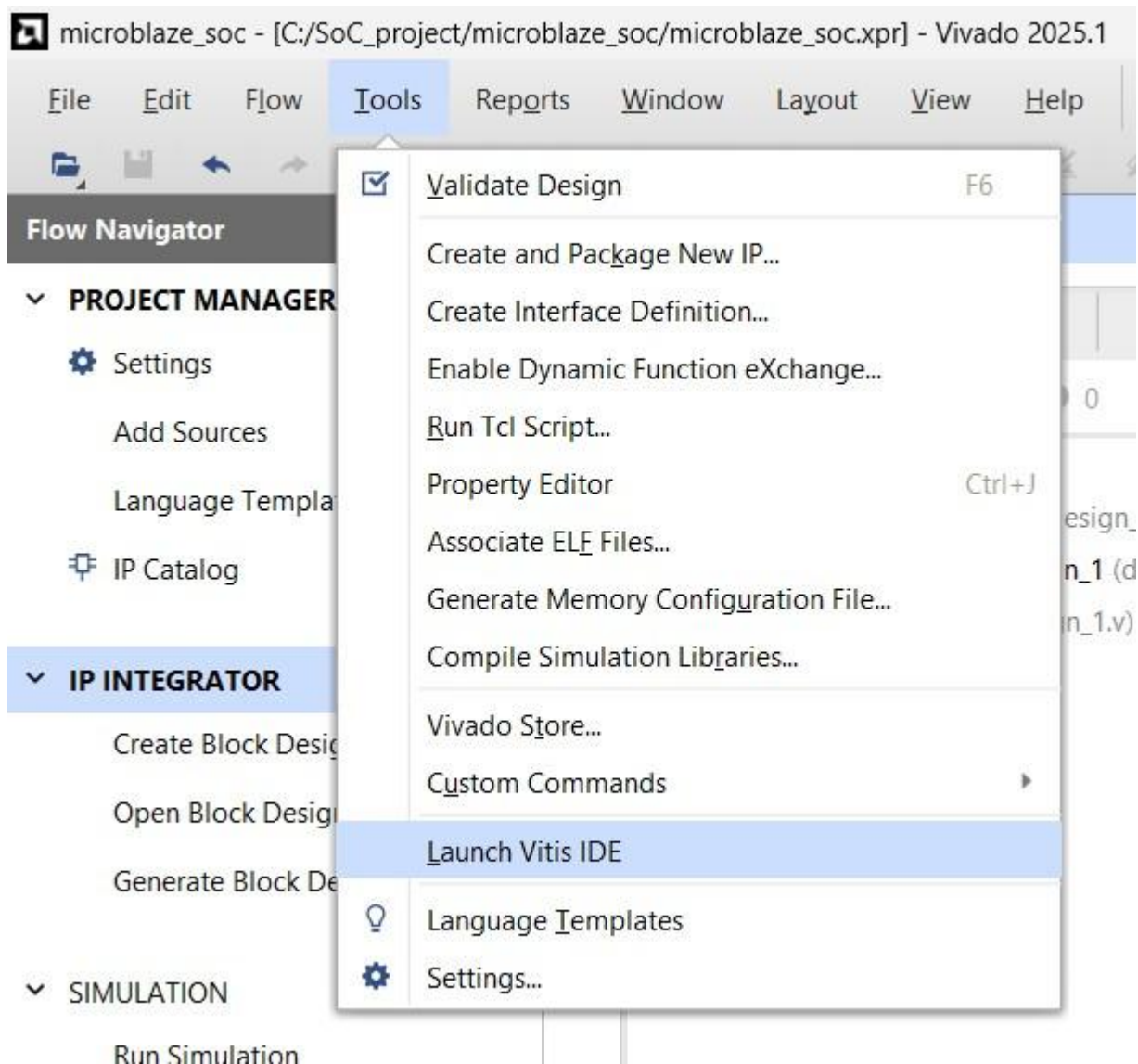
5. Export the Hardware Design to Vitis

Once the bitstream is generated, export the design to Vitis where the software interface program is written. This program sends mode commands to the custom LED IP and controls the hardware behavior.

1. First, make sure the ZedBoard is powered on and connected via JTAG.
2. In Vivado, go to the Flow Navigator and click **Open Hardware Manager** → **Open Target** → **Open New Target**. The New Hardware Target wizard appears.
3. Click Next, select the hardware target (the ZedBoard should appear), click Next, then Finish.
4. Click **Refresh Device**, then click **Program Device**. A dialog appears.
5. Select the generated bitstream file and click **Program**. The FPGA is now programmed with the hardware design.
6. From the Vivado file menu, select **File** → **Export** → **Export Hardware**.



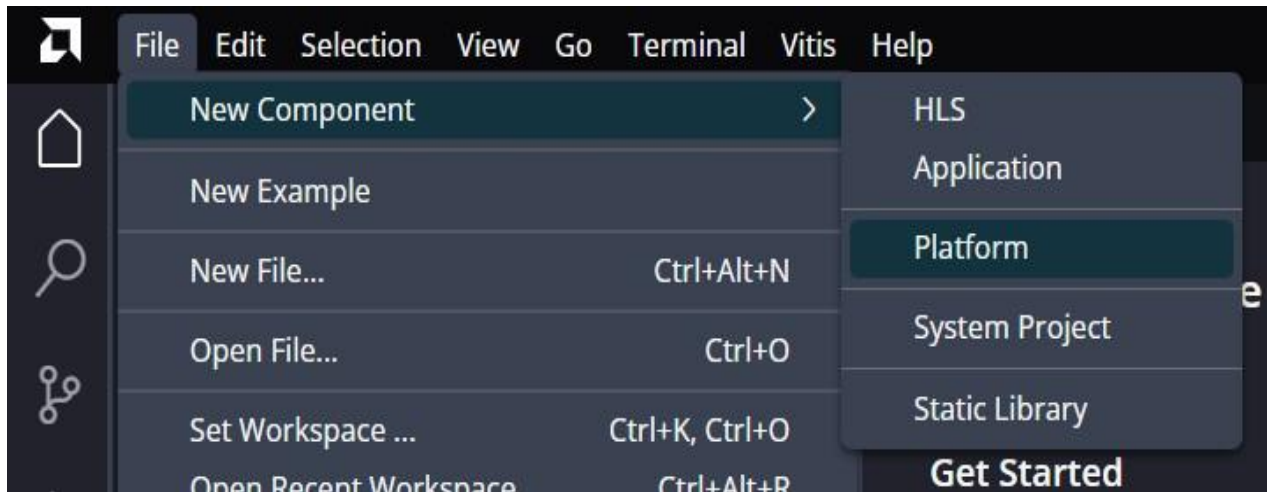
7. In the window that appears, select "**Fixed**" and click "Next".
8. Select "**Include Bitstream**" and click "Next".
9. Browse to the desired folder, give the file a name, and click "Next" then "Finish". A .xsa file is created containing the hardware description and bitstream.
10. From the Tools menu, select "**Launch Vitis IDE**".



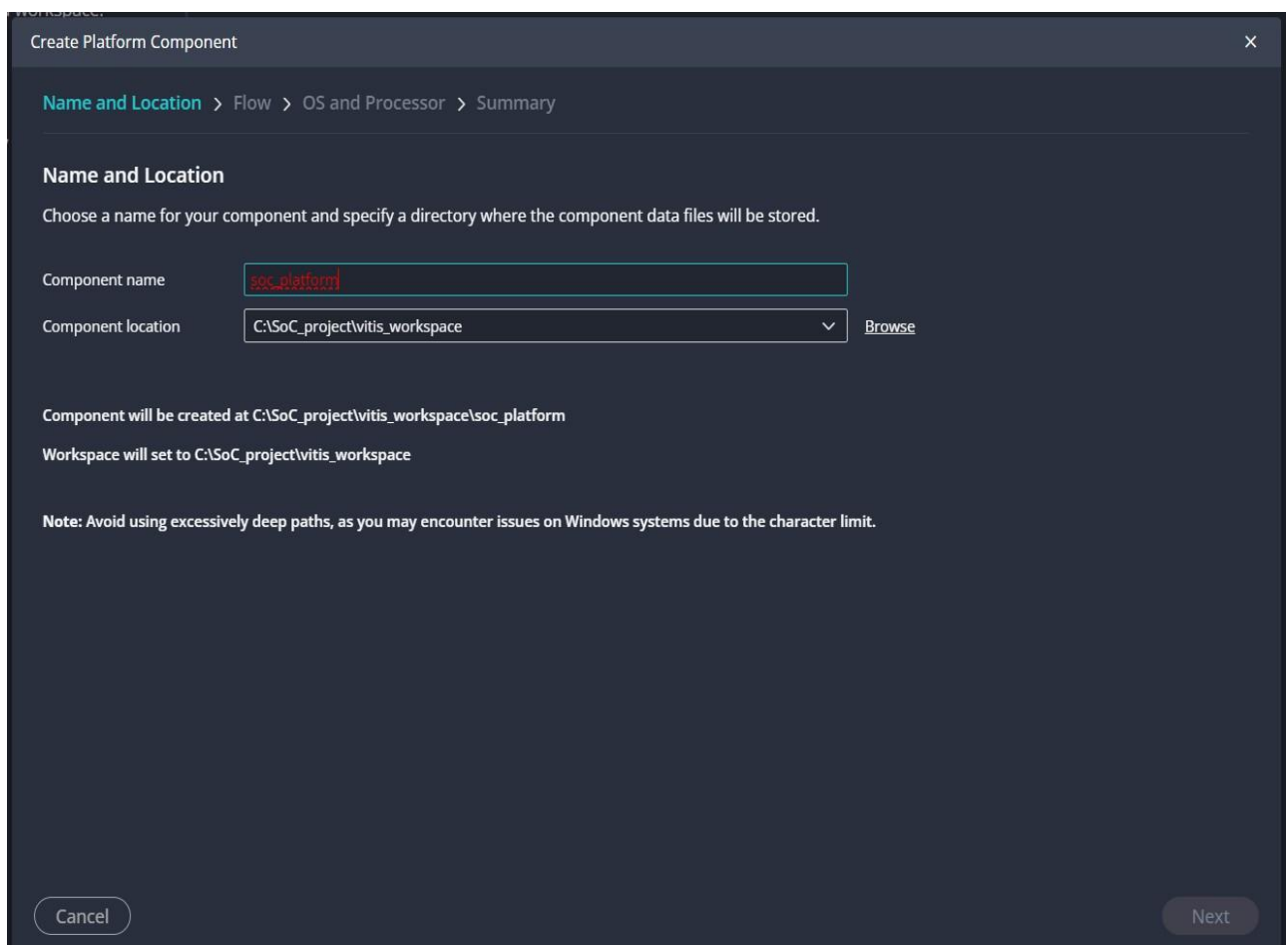
11. In the workspace selection window, browse to a workspace directory and click "Launch".

6. Create the Software Application

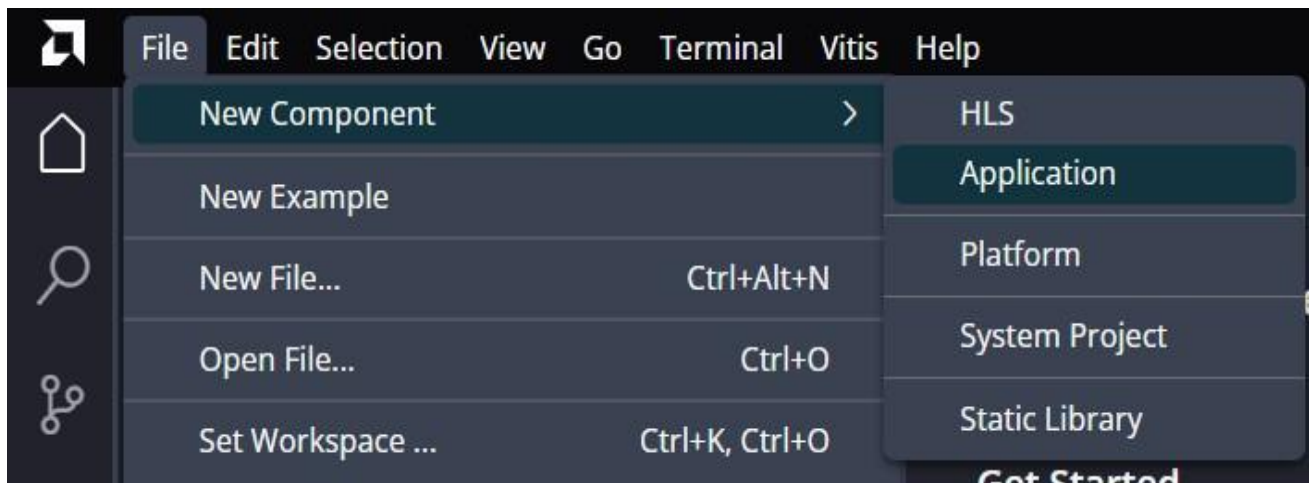
1. In Vitis, click **File** → **New** → **Application Project** (or use Create Application Component in Vitis 2025.1).



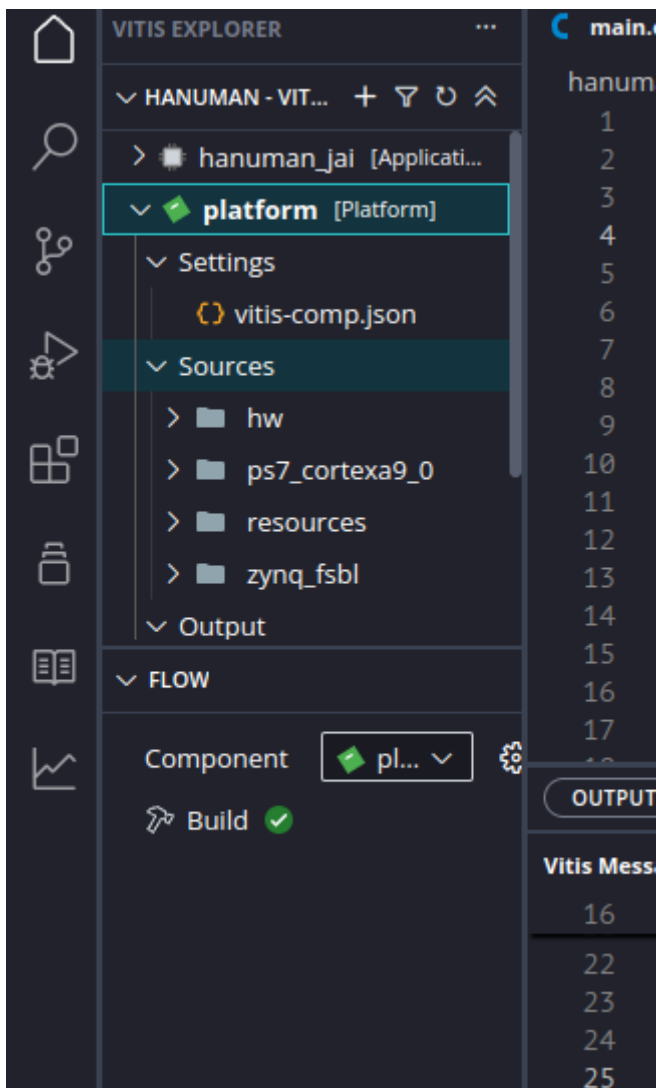
2. In the window that appears, click "Next".
3. Click "**Create a new platform from hardware (XSA)**", browse to the .xsa file exported from Vivado, and click "Next".



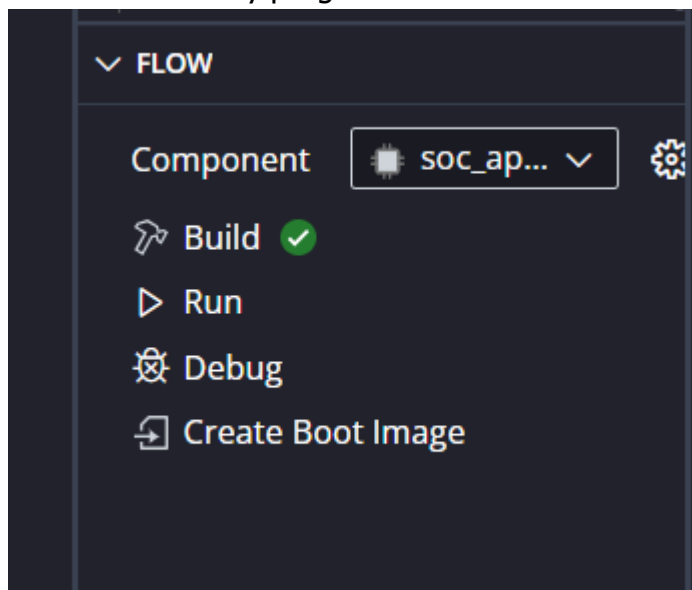
4. Type the **Application Project Name** as accel_app, select the MicroBlaze processor, and click "Next".



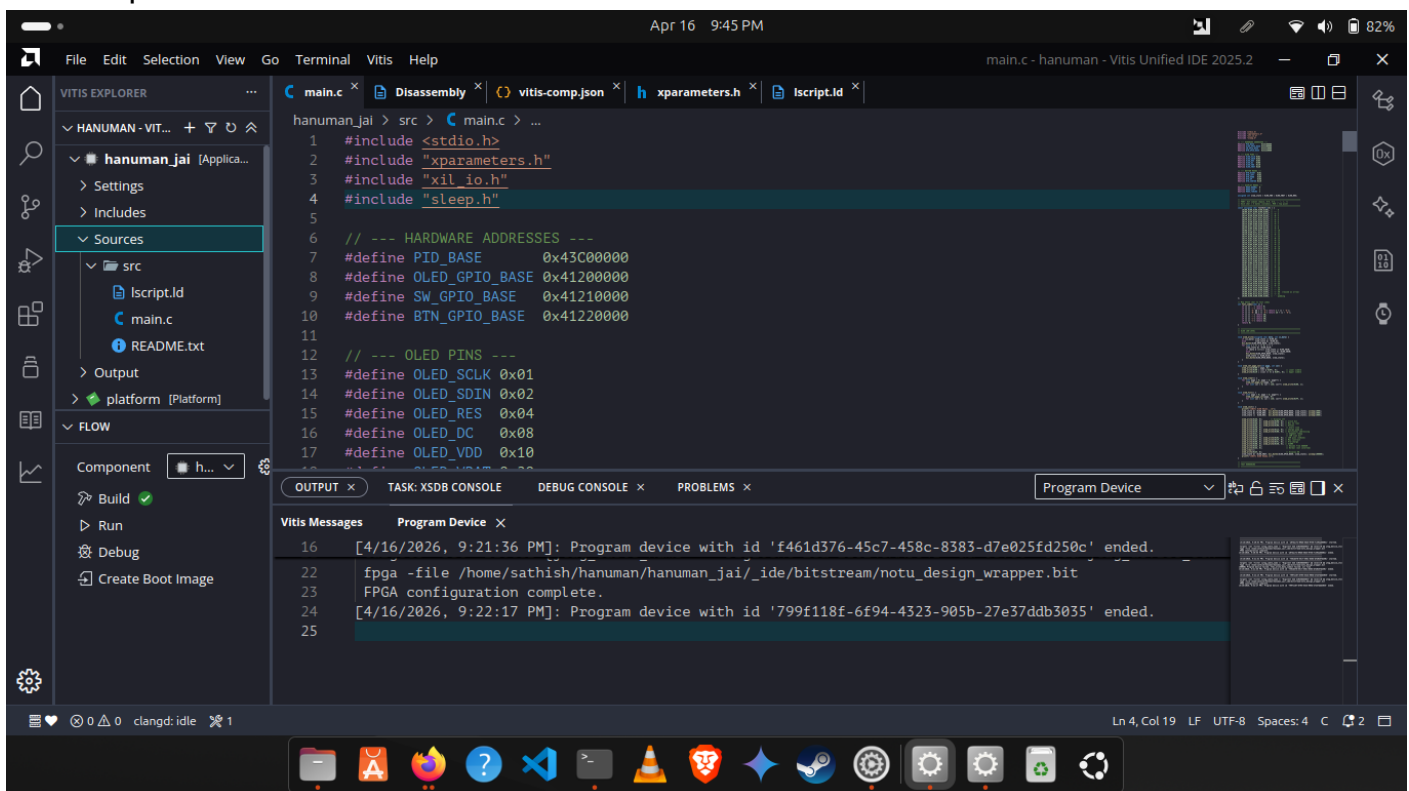
5. In the template window, select "**microblaze_code**" and click "Finish".
6. The project files are added to the Project Explorer window, including the main C source file.
7. Double-click the main .c file and replace its contents with the LED accelerator program.
8. After editing the program, right-click the project folder and select "**Build Project**" to compile.



9. After the project is built, right-click the project folder and select **Run As** → **Launch on Hardware**. Vitis automatically programs the FPGA and loads the executable into the MicroBlaze BRAM.



10. The output appears on JTAG terminal. The main menu is displayed and the system waits for a key input.



11. The OLED BEHAVIOUR on the board changes according to the PUSH BUTTON selected.

Thus the command has been successfully sent from the PC to the MicroBlaze processor, which writes to the custom IP registers, and the hardware responds by driving the LEDs accordingly.


```
Vitis Messages Program Device X
16 [4/16/2026, 9:21:36 PM]: Program device with id 'f461d376-45c7-458c-8383-d7e025fd250c' ended.
17 -----
18 -----
19 [4/16/2026, 9:22:14 PM]: Program device with id '799f118f-6f94-4323-905b-27e37ddb3035' started.
20 -----
21 targets -set -filter {jtag_cable_name =~ "Digilent Zed 210248AA5841" && level==0 && jtag_device_ctx==
22 fpga -file /home/sathish/hanuman/hanuman_jai/_ide/bitstream/notu_design_wrapper.bit
23 FPGA configuration complete.
24 [4/16/2026, 9:22:17 PM]: Program device with id '799f118f-6f94-4323-905b-27e37ddb3035' ended.
25 -----
```

Software Application Program

```
#include <stdio.h>
```

```
#include "xparameters.h"
```

```
#include "xil_io.h"
```

```
#include "sleep.h"
```

```
// --- HARDWARE ADDRESSES ---
```

```
#define PID_BASE 0x43C00000
```

```
#define OLED_GPIO_BASE 0x41200000
```

```
#define SW_GPIO_BASE 0x41210000
```

```
#define BTN_GPIO_BASE 0x41220000
```

```
// --- OLED PINS ---
```

```
#define OLED_SCLK 0x01
```

```
#define OLED_SDIN 0x02
```

```
#define OLED_RES 0x04
```

```
#define OLED_DC 0x08
```

```
#define OLED_VDD 0x10
```

```
#define OLED_VBAT 0x20
```

```
// --- BUTTON MASKS ---
```

```
#define BTN_RIGHT 0x01
```

```
#define BTN_LEFT 0x02
```

```
#define BTN_DOWN 0x04
```

```
#define BTN_UP 0x08
```

```
#define BTN_CENTER 0x10
```

```
// --- DISPLAY MODES ---
```

```
#define MODE_NORMAL 0
```

```
#define MODE_STOP 1
```

```
#define MODE_TUNING 2
```

```
unsigned int oled_state = OLED_VDD | OLED_VBAT | OLED_RES;
```

```
//
```

```
=====
=====
```

```
// FONT: 5x7 subset (Space, 0-9, A-Z, :, -, >, !)
```

```
// Each char = 5 bytes (columns), MSB = top pixel
```

```
//
```

```
=====
```



```

const unsigned char font5x7[][5] = {
{0x00,0x00,0x00,0x00,0x00}, // ' ' 0
{0x00,0x5F,0x00,0x00,0x00}, // '!' 1
{0x7C,0x12,0x11,0x12,0x7C}, // 'A' 2
{0x7F,0x49,0x49,0x49,0x36}, // 'B' 3
{0x3E,0x41,0x41,0x41,0x22}, // 'C' 4
{0x7F,0x41,0x41,0x22,0x1C}, // 'D' 5
{0x7F,0x49,0x49,0x49,0x41}, // 'E' 6
{0x7F,0x09,0x09,0x09,0x01}, // 'F' 7
{0x3E,0x41,0x49,0x49,0x7A}, // 'G' 8
{0x7F,0x08,0x08,0x08,0x7F}, // 'H' 9
{0x00,0x41,0x7F,0x41,0x00}, // 'I' 10
{0x20,0x40,0x41,0x3F,0x01}, // 'J' 11
{0x7F,0x08,0x14,0x22,0x41}, // 'K' 12
{0x7F,0x40,0x40,0x40,0x40}, // 'L' 13
{0x7F,0x02,0x0C,0x02,0x7F}, // 'M' 14
{0x7F,0x04,0x08,0x10,0x7F}, // 'N' 15
{0x3E,0x41,0x41,0x41,0x3E}, // 'O' 16
{0x7F,0x09,0x09,0x09,0x06}, // 'P' 17
{0x3E,0x41,0x51,0x21,0x5E}, // 'Q' 18
{0x7F,0x09,0x19,0x29,0x46}, // 'R' 19
{0x46,0x49,0x49,0x49,0x31}, // 'S' 20
{0x01,0x01,0x7F,0x01,0x01}, // 'T' 21
{0x3F,0x40,0x40,0x40,0x3F}, // 'U' 22
{0x1F,0x20,0x40,0x20,0x1F}, // 'V' 23
{0x3F,0x40,0x38,0x40,0x3F}, // 'W' 24
{0x63,0x14,0x08,0x14,0x63}, // 'X' 25
{0x07,0x08,0x70,0x08,0x07}, // 'Y' 26
{0x61,0x51,0x49,0x45,0x43}, // 'Z' 27
{0x3E,0x51,0x49,0x45,0x3E}, // 'o' 28
{0x00,0x42,0x7F,0x40,0x00}, // 'l' 29
{0x42,0x61,0x51,0x49,0x46}, // '2' 30
{0x21,0x41,0x45,0x4B,0x31}, // '3' 31
{0x18,0x14,0x12,0x7F,0x10}, // '4' 32
{0x27,0x45,0x45,0x45,0x39}, // '5' 33
{0x3C,0x4A,0x49,0x49,0x30}, // '6' 34
{0x01,0x71,0x09,0x05,0x03}, // '7' 35
{0x36,0x49,0x49,0x49,0x36}, // '8' 36
{0x06,0x49,0x49,0x29,0x1E}, // '9' 37
{0x00,0x36,0x36,0x00,0x00}, // ':' 38
{0x08,0x08,0x08,0x08,0x08}, // '-' 39
{0x00,0x41,0x22,0x14,0x08}, // '<' 40 (reused as arrow)
{0x00,0x08,0x14,0x22,0x41}, // '>' 41
{0x00,0x00,0x00,0x00,0x00}, // ' ' padding
};

```

// Map ASCII char to font index

```

int font_index(char c) {
if (c == ' ') return 0;
if (c == '!') return 1;
if (c >= 'A' && c <= 'Z') return 2 + (c - 'A');
if (c >= '0' && c <= '9') return 28 + (c - '0');
if (c == ':') return 38;
if (c == '-') return 39;
}

```

```

if (c == '<') return 40;
if (c == '>') return 41;
return 0;
}

//
=====
=====
// OLED LOW-LEVEL
//
=====
=====

void oled_write(unsigned char data, int is_data) {
if (is_data) oled_state |= OLED_DC;
else oled_state &= ~OLED_DC;
Xil_Out32(OLED_GPIO_BASE, oled_state);
for (int i = 7; i >= 0; i--) {
oled_state &= ~OLED_SCLK;
if (data & (1<<i)) oled_state |= OLED_SDIN;
else oled_state &= ~OLED_SDIN;
Xil_Out32(OLED_GPIO_BASE, oled_state);
oled_state |= OLED_SCLK;
Xil_Out32(OLED_GPIO_BASE, oled_state);
}
}

void oled_set_page_col(int page, int col) {
oled_write(0xB0 + page, 0);
oled_write(0x00 | (col & 0x0F), 0); // Lower nibble
oled_write(0x10 | ((col >> 4) & 0x0F), 0); // Upper nibble
}

void oled_clear() {
for (int page = 0; page < 4; page++) {
oled_set_page_col(page, 0);
for (int col = 0; col < 128; col++) oled_write(0x00, 1);
}
}

void oled_fill() {
for (int page = 0; page < 4; page++) {
oled_set_page_col(page, 0);
for (int col = 0; col < 128; col++) oled_write(0xFF, 1);
}
}

void oled_init() {
printf("[INIT] OLED Reset...\n");
oled_state &= ~OLED_VDD; Xil_Out32(OLED_GPIO_BASE, oled_state); usleep(1000);
oled_state &= ~OLED_RES; Xil_Out32(OLED_GPIO_BASE, oled_state); usleep(1000);
oled_state |= OLED_RES; Xil_Out32(OLED_GPIO_BASE, oled_state); usleep(1000);

oled_write(0xAE, 0); // Display off

```

```

oled_write(0xD5, 0); oled_write(0x80, 0); // Clock div
oled_write(0xA8, 0); oled_write(0x1F, 0); // Mux 32 rows
oled_write(0xD3, 0); oled_write(0x00, 0); // Offset
oled_write(0x40, 0); // Start line
oled_write(0x8D, 0); oled_write(0x14, 0); // Charge pump on
oled_write(0x20, 0); oled_write(0x00, 0); // Horizontal addressing
oled_write(0xA1, 0); // Segment remap
oled_write(0xC8, 0); // COM scan dir
oled_write(0xDA, 0); oled_write(0x02, 0); // COM pins (128x32)
oled_write(0x81, 0); oled_write(0xCF, 0); // Max contrast
oled_write(0xD9, 0); oled_write(0xF1, 0); // Pre-charge
oled_write(0xDB, 0); oled_write(0x40, 0); // VCOMH
oled_write(0xA4, 0); // Resume from GDDRAM
oled_write(0xA6, 0); // Normal (not inverted)
oled_clear();
oled_write(0xAF, 0); // Display ON
oled_state &= ~OLED_VBAT; Xil_Out32(OLED_GPIO_BASE, oled_state);
usleep(100000);
printf("[INIT] OLED Ready.\n");
}

//
=====
====
// TEXT RENDERING
//
=====
=====

// Draw one char at (page, col). Returns next col position.
// invert=1 → white background black text (fake "highlight")
int oled_draw_char(int page, int col, char c, int invert) {
int idx = font_index(c);
for (int i = 0; i < 5; i++) {
unsigned char byte = font5x7[idx][i];
oled_set_page_col(page, col + i);
oled_write(invert ? ~byte : byte, 1);
}
// 1px gap
oled_set_page_col(page, col + 5);
oled_write(invert ? 0xFF : 0x00, 1);
return col + 6;
}

// Draw string at (page, col). Clip at 128.
int oled_draw_string(int page, int col, const char* str, int invert) {
while (*str && col < 122) {
col = oled_draw_char(page, col, *str++, invert);
}
return col;
}

// Draw integer as string
void oled_draw_int(int page, int col, int val, int digits, int invert) {

```

```

char buf[12];
int i = digits - 1;
int neg = (val < 0);
if (neg) val = -val;
buf[digits] = '\0';
while (i >= 0) {
    buf[i--] = '0' + (val % 10);
    val /= 10;
}
if (neg) buf[0] = '-';
oled_draw_string(page, col, buf, invert);
}

//
=====
=====
// DISPLAY LAYOUTS
//
=====
=====

// "Visual pattern" bar — uses checkerboard for empty region
// so it looks more structured than plain black
void oled_draw_gauge_fancy(int speed, int target, int mode) {
    if (speed < 0) speed = 0;
    if (speed > 255) speed = 255;
    if (target < 0) target = 0;
    if (target > 255) target = 255;

    int bar = (speed * 120) / 255; // 0..120 px filled
    int tgt_px = (target * 120) / 255; // target marker position

    // --- PAGE 0: Speed bar with target marker ---
    oled_set_page_col(0, 0);
    for (int col = 0; col < 128; col++) {
        unsigned char px;
        if (col == 0 || col == 127) {
            px = 0xFF; // Side border
        } else if (col == tgt_px + 3) {
            px = 0b10101010; // Target marker: dotted line
        } else if (col - 3 <= bar) {
            // Filled region: solid top, lighter bottom
            px = (mode == MODE_STOP) ? 0b00011000 : 0xFF;
        } else {
            // Empty region: subtle dot pattern
            px = (col % 4 == 0) ? 0b00010000 : 0x00;
        }
        oled_write(px, 1);
    }

    // --- PAGE 1: Thin bar bottom edge + separator ---
    oled_set_page_col(1, 0);
    for (int col = 0; col < 128; col++) {
        unsigned char px;

```

```

if (col == 0 || col == 127) {
    px = 0xFF;
} else if (col - 3 <= bar) {
    px = (mode == MODE_STOP) ? 0b00000011 : 0b00000111;
} else {
    px = (col % 4 == 0) ? 0b00000001 : 0x00;
}
oled_write(px, 1);
}

// --- PAGE 2: Text row: "SPD:NNN TGT:NNN" ---
// Clear page first
oled_set_page_col(2, 0);
for (int col = 0; col < 128; col++) oled_write(0x00, 1);

if (mode == MODE_STOP) {
    // Inverted "STOP" banner
    oled_set_page_col(2, 0);
    for (int col = 0; col < 128; col++) oled_write(0xFF, 1); // full white bg
    oled_draw_string(2, 30, "EMERGENCY STOP", 1); // black text on white
} else if (mode == MODE_TUNING) {
    oled_draw_string(2, 0, "TUNE KP:", 0);
} else {
    oled_draw_string(2, 0, "SPD:", 0);
    oled_draw_int(2, 26, speed, 3, 0);
    oled_draw_string(2, 48, "TGT:", 0);
    oled_draw_int(2, 74, target, 3, 0);
}

// --- PAGE 3: Status row ---
oled_set_page_col(3, 0);
for (int col = 0; col < 128; col++) oled_write(0x00, 1);

if (mode == MODE_TUNING) {
    // Show KP value large
    oled_draw_string(3, 0, "KP:", 0);
} else {
    // Scrolling indicator dots (loop counter trick)
    oled_draw_string(3, 0, "CRUISE CTRL", 0);
}

// Full-screen blink for emergency stop
void oled_blink(int times) {
    for (int i = 0; i < times; i++) {
        oled_fill(); usleep(150000);
        oled_clear(); usleep(150000);
    }
}

//
=====
====
// SAFE PID READ

```

```

//
=====
=====

int pid_compute(int kp, int ki, int kd, int err, int intg, int derv) {
Xil_Out32(PID_BASE + 0, (u32)kp);
Xil_Out32(PID_BASE + 4, (u32)ki);
Xil_Out32(PID_BASE + 8, (u32)kd);
Xil_Out32(PID_BASE + 12, (u32)err);
Xil_Out32(PID_BASE + 16, (u32)intg);
Xil_Out32(PID_BASE + 20, (u32)derv);
usleep(10);
return (int)Xil_In32(PID_BASE + 24);
}

//
=====
=====
// BUTTON DEBOUNCE
//
=====
=====

unsigned int btn_prev = 0;

// Returns newly pressed buttons (rising edge only)
unsigned int btn_read_edge() {
unsigned int now = Xil_In32(BTN_GPIO_BASE) & 0x1F;
unsigned int edge = now & ~btn_prev; // only newly pressed
btn_prev = now;
return edge;
}

//
=====
=====
// MAIN
//
=====
=====

int main() {
printf("\033[2J\033[H");
printf("===== \n");
printf(" CRUISE CONTROL — ARVIND PRAKASH\n");
printf(" BTN: UP/DN=KP | LR=Speed | CTR=STOP\n");
printf("===== \n\n");

oled_init();

// Startup animation: sweep bar
for (int s = 0; s <= 255; s += 8) {

```

```

oled_draw_gauge_fancy(s, 255, MODE_NORMAL);
usleep(15000);
}
for (int s = 255; s >= 0; s -= 8) {
oled_draw_gauge_fancy(s, 0, MODE_NORMAL);
usleep(15000);
}

int kp = 3, ki = 1, kd = 2;
int current_speed = 0, target_speed = 0;
int error = 0, prev_error = 0, integral = 0, derivative = 0;
int control_effort = 0;
int mode = MODE_NORMAL;
int loop = 0;
int manual_override = 0; // Set when LEFT/RIGHT used

printf("[READY] Loop starting...\n\n");

while (1) {
loop++;

// --- READ BUTTONS (edge-triggered) ---
unsigned int btns = btn_read_edge();

// CENTER: Emergency Stop
if (btns & BTN_CENTER) {
mode = MODE_STOP;
current_speed = 0;
integral = 0;
printf("[BTN-CENTER] Emergency Stop! Loop=%d\n", loop);
oled_blink(3);
}

// UP: Increase KP
if (btns & BTN_UP) {
if (kp < 20) kp++;
mode = MODE_TUNING;
printf("[BTN-UP] KP increased to %d\n", kp);
}

// DOWN: Decrease KP
if (btns & BTN_DOWN) {
if (kp > 0) kp--;
mode = MODE_TUNING;
printf("[BTN-DOWN] KP decreased to %d\n", kp);
}

// RIGHT: Manual speed up
if (btns & BTN_RIGHT) {
target_speed += 16;
if (target_speed > 255) target_speed = 255;
manual_override = 1;
if (mode != MODE_STOP) mode = MODE_NORMAL;
printf("[BTN-RIGHT] Target speed -> %d\n", target_speed);
}

```

```

}

// LEFT: Manual speed down
if (btns & BTN_LEFT) {
    target_speed -= 16;
    if (target_speed < 0) target_speed = 0;
    manual_override = 1;
    if (mode != MODE_STOP) mode = MODE_NORMAL;
    printf("[BTN-LEFT] Target speed -> %d\n", target_speed);
}

// If not manually overriding, read switches for target
if (!manual_override) {
    target_speed = (int)(Xil_In32(SW_GPIO_BASE) & 0xFF);
}

// Resume from STOP after CENTER press + any other button
if (mode == MODE_STOP && (btns & ~BTN_CENTER)) {
    mode = MODE_NORMAL;
    printf("[RESUME] Exiting stop mode.\n");
}

// Auto-exit tuning mode after no button activity (show normal)
if (mode == MODE_TUNING && btns == 0) {
    // keep tuning display for 5 loops then auto-revert
    static int tune_timer = 0;
    tune_timer++;
    if (tune_timer > 5) { mode = MODE_NORMAL; tune_timer = 0; }
}

// --- PID (skip if stopped) ---
if (mode != MODE_STOP) {
    error = target_speed - current_speed;
    integral += error;
    if (integral > 200) integral = 200;
    if (integral < -200) integral = -200;
    derivative = error - prev_error;

    control_effort = pid_compute(kp, ki, kd, error, integral, derivative);

    current_speed += (control_effort / 16) - 4;
    if (current_speed < 0) current_speed = 0;
    if (current_speed > 255) current_speed = 255;
}
prev_error = error;

// --- DISPLAY ---
oled_draw_gauge_fancy(current_speed, target_speed, mode);

// --- UART LOG ---
printf("[%04d] SPD:%03d TGT:%03d EFFORT:%06d KP:%0d MODE:%0d\n",
loop, current_speed, target_speed, control_effort, kp, mode);

usleep(2000000);}

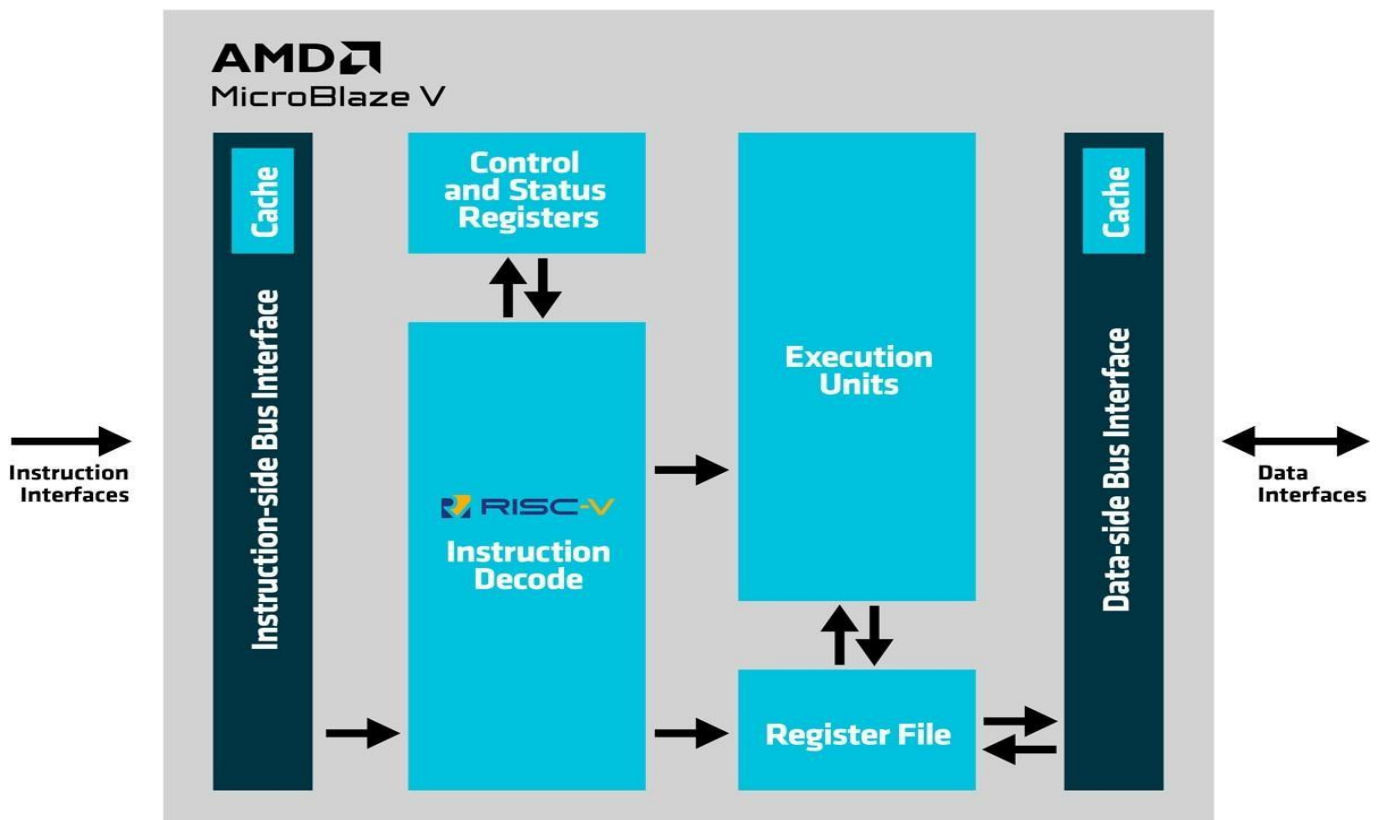
```



```
return 0;  
}
```

ABOUT MICROBLAZE V

MicroBlaze V Overview Illustration



AMD MicroBlaze V Processor

The AMD MicroBlaze V processor is a soft-core **RISC-V based** processor IP for AMD FPGAs and adaptive SoCs. It is built on the RISC-V open-source instruction set architecture (ISA), managed by the non-profit RISC-V Foundation — of which AMD has been a member since 2020.

It is hardware compatible with the classic MicroBlaze processor and fully integrated into the Vivado and Vitis design flow, available at no extra cost for any supported AMD device.

Key Features:

- Supports RV32I and RV64I base integer instruction sets with optional M, A, F, C, and B extensions
- Multiple pipeline options to balance area and performance
- Optional dual-core lockstep and triple modular redundancy (TMR) for safety-critical systems
- Configurable presets from simple microcontroller to full application processor
- Fully integrated with Vivado and Vitis tools
- Compatible with all AMD adaptive SoCs and FPGAs

The MicroBlaze V processor is highly modular and configurable, making it ideal for embedded SoC designs like this project.

Summary

The **Hardware-Accelerated Closed-Loop PID Control System** is a sophisticated embedded platform that demonstrates **Hardware-Software Co-Design** on the Zynq-7000 SoC. By offloading the PID control laws to custom FPGA logic, the system achieves real-time, deterministic performance that software alone cannot match.

The project successfully bridges high-level user interaction (Buttons/OLED) with low-level hardware acceleration (AXI-mapped PID Core) to create a reliable Cruise Control simulation. Key achievements include the resolution of peripheral timing constraints, the implementation of a robust 12V power distribution strategy, and the development of a clean, glitch-free visual telemetry system.

Final Identification Table:

Component	Specification
Logic Platform	ZedBoard Zynq-7000 (PL + PS)
Control Logic	Custom Hardware-Mapped PID IP
Sampling Rate	50ms (Internal Loop)
Visual Output	128x32 SSD1306 OLED (Bit-banged)
Safety Protocol	Asynchronous Center-Button Interrupt
Power Supply	12V External DC (to prevent brownouts)